



北京化工大学

硕士学位论文

题 目 面向在线评测系统的测试数据评估与生成技术
研究

专 业 计算机科学与技术

研 究 生 杨泽忠

指导教师 刘勇

日期：20250527

学位论文数据集

中图分类号	TP311.5	学科分类号	520.4070	
论文编号	1001020250804	密 级	公开	
学位授予 单位代码	10010	学位授予 单位名称	北京化工大学	
作者姓名	杨泽忠	学 号	2022200804	
获学位专业 名称	计算机科学与技术	获学位专业 代码	081200	
课题来源	国家自然科学基金	研究方向	软件测试	
论文题目	面向在线评测系统的测试数据评估与生成技术研究			
关 键 词	在线评测系统，差分测试，测试数据生成，大语言模型，多目标优化算法			
论文答辩日期	2025 年 5 月 23 日	论 文 类 型	基础研究	
学位论文评阅及答辩委员会情况				
	姓名	职称	工作单位	学科专长
指导教师	刘勇	教授	北京化工大学	软件工程
评阅人 1	盲审			
评阅人 2	盲审			
评阅人 3	盲审			
答辩委员会主席	耿志强	教授	北京化工大学	工业大数据 与机器学习
答辩委员 1	赵英	教授	北京化工大学	计算机网络
答辩委员 2	林晓勇	副教授	北京化工大学	数据挖掘
答辩委员 3	江志英	高工	北京化工大学	大数据实时 分析处理
答辩委员 4	李辉	副教授	北京化工大学	密码学、 人工智能

面向在线评测系统的测试数据评估与生成技术研究

摘 要

在线评测系统（Online Judge System, OJ）作为编程教育和竞赛的重要平台，依赖测试数据对学生程序进行自动评判。目前 OJ 系统中测试数据的设计以人工构造为主，费时费力且难以全面覆盖复杂场景，导致出现漏报情况，即将有缺陷的程序评判为正确程序（假阳性程序）。因此，如何评估现有 OJ 系统中测试数据的质量以及自动生成高质量测试数据成为了研究重点。然而，目前关于测试数据质量评估与生成的研究主要面向工业程序，未能充分结合 OJ 系统中学生程序的特点进行优化，因而其适用性和效果在 OJ 场景中表现不佳。

针对上述问题，本文首先提出了一种面向 OJ 系统的测试数据评估框架，基于代码行覆盖率、分支覆盖率、变异得分以及专为 OJ 系统设计的测试规模，对现有测试数据的质量进行多维评估。为检测 OJ 系统中的漏报情况，本文提出了一种基于聚类和差分测试的假阳性程序检测方法。该方法利用学生程序的动态特征进行聚类，约简高度相似的学生程序，并基于题目信息和程序静态特征生成测试数据，降低差分测试的执行成本。其次，本文提出了一种基于大语言模型和多目标优化算法的测试数据生成方法，通过基于思维链（Chain of Thought, COT）的提示词工程引导大语言模型生成多样化的测试数据，并以此作为初始种群，使用多目标优化算法进行迭代优化。在优化过程中，以错误检测率、代码覆盖路径和测试规模

的加权和为目标，引导测试数据的生成方向，最终生成兼顾错误检测能力和测试规模的高质量测试数据。

为了评估所提方法的有效性，本文基于 BuctOJ 和 AtCoder 两个在线编程平台，构建了一个包含 33205 份学生程序的数据集，并在此基础上进行了实验研究。实验结果显示，现有测试数据的平均行覆盖率仅为 81.71%，分支覆盖率为 79.35%，变异得分为 0.81，错误检测能力较低。在假阳性程序检测方面，与最新发表的差分测试（differential testing，DT）方法相比，本文方法在检测能力相同的前提下，减少了约 50.39%的时间开销。在测试数据生成方面，相较于传统人工设计和基于遗传算法的方法，本文方法生成的测试数据的错误检测率分别提升了 6.07%和 6.77%，测试规模分别缩小了 6.64 倍和 5.05 倍，假阳性程序检测率较最新提出的 AID 方法提升了 19.44%。

基于上述研究成果，本文设计并实现了测试数据评估与生成工具原型 TestEG，并详细阐述了其核心模块设计与系统实现，为编程教育提供了有效的辅助支持。

关键词：在线评测系统，差分测试，测试数据生成，大语言模型，多目标优化算法

RESEARCH ON TEST DATA EVALUATION AND GENERATION TECHNIQUES FOR ONLINE JUDGE SYSTEM

ABSTRACT

The Online Judge System (OJ), as critical platform for programming education and competitions, relies on test data to automatically evaluate student programs. Currently, the design of test data in the OJ system primarily depends on manual construction, which is time-consuming, labor-intensive, and fails to comprehensively cover complex scenarios, leading to false negatives, where defective programs (false positive programs) are incorrectly judged as correct. Therefore, evaluating the quality of existing test data in the OJ system and automatically generating high-quality test data have become key research focuses. However, existing studies on test data quality evaluation and generation mainly target industrial programs and fail to adequately address the specific characteristics of student programs in the OJ system, resulting in limited applicability and effectiveness in this context.

To address these issues, this paper first proposes a test data evaluation framework tailored for the OJ system, evaluating the quality of existing test data from multiple dimensions, including line coverage, branch coverage, mutation

score, and test scale metric designed specifically for the OJ system. To detect false negatives in the OJ system, this paper proposes a false positive program detection approach based on clustering and differential testing. The approach clusters student programs using dynamic features to reduce programs that are highly similar and generates test data based on problem information and static program features, thereby lowering the execution cost of differential testing. Additionally, this paper proposes a test data generation approach combining large language models and multi-objective optimization algorithms. By employing prompt engineering based on Chain of Thought (COT), the large language model is guided to generate diverse test data as an initial population, which is then iteratively optimized using a multi-objective optimization algorithm. During optimization, the weighted sum of faulty code detection rate, code coverage paths, and test scale serves as the objective, directing the generation of high-quality test data that balances fault detection capability and test scale.

To evaluate the effectiveness of the proposed approaches, this study constructs a dataset containing 33,205 student programs from two online programming platforms, BuctOJ and AtCoder, and conducts experimental research based on this dataset. The results show that the average line coverage of existing test data is only 81.71%, with a branch coverage of 79.35% and a mutation score of 0.81, indicating a low fault detection capability. In false positive detection, compared to the state-of-the-art differential testing approach,

the proposed approach achieves equivalent detection performance while reducing time overhead by approximately 50.39%. In test data generation, compared to traditional manual design and genetic algorithm-based approaches, the proposed approach improves fault detection rates by 6.07% and 6.77%, reduces test scale by 6.64 times and 5.05 times, and increases false positive detection rates by 19.44% compared to the recently proposed AID approach.

Based on these findings, this paper designs and implements a prototype tool, TestEG, for test data evaluation and generation, detailing its core module design and system implementation. This tool provides effective support for programming education.

KEY WORDS: online judge system, differential testing, test data generation, large language model, multi-objective optimization algorithm

目 录

第一章 绪论.....	1
1.1 课题研究背景和意义.....	1
1.2 研究现状.....	2
1.2.1 测试数据评估.....	2
1.2.2 测试数据生成.....	3
1.3 课题主要研究内容.....	4
1.4 本文组织结构.....	5
第二章 背景和相关工作	7
2.1 在线评测系统.....	7
2.2 多目标优化算法.....	8
2.2.1 非支配排序遗传算法.....	8
2.2.2 强度帕累托进化算法.....	9
2.3 测试数据评估方法.....	9
2.4 测试数据生成方法.....	10
2.4.1 基于符号执行的测试数据生成.....	10
2.4.2 基于搜索算法的测试数据生成.....	11
2.4.3 基于大语言模型的测试数据生成.....	11
2.5 本章小结.....	11
第三章 基于聚类与差分测试的测试数据评估方法研究	13
3.1 研究动机.....	13
3.2 在线评测系统测试数据评估框架	15
3.2.1 代码覆盖.....	15
3.2.2 错误检测.....	16
3.2.3 测试规模.....	17
3.3 基于程序动态和静态特征的假阳性程序检测方法	18
3.3.1 基于动态特征的学生程序聚类.....	18
3.3.2 基于静态特征的差分测试.....	21
3.4 实验设计.....	23
3.4.1 研究问题.....	23
3.4.2 实验对象.....	24
3.4.3 评价指标.....	25
3.4.4 基准方法.....	26
3.5 结果分析.....	27
3.5.1 在线评测系统现有测试数据质量分析	27

3.5.2 本文方法的有效性分析.....	31
3.5.3 本文方法与基准方法的效果对比.....	32
3.6 本章小结.....	34
第四章 基于大语言模型和多目标优化算法的测试数据生成技术	35
4.1 研究动机.....	35
4.2 基于大语言模型和多目标优化算法的测试数据生成方法	35
4.2.1 大语言模型预生成测试数据.....	36
4.2.2 多目标优化算法持续优化.....	38
4.3 实验设计.....	44
4.3.1 研究问题.....	44
4.3.2 评价指标.....	45
4.3.3 基准方法.....	45
4.4 结果分析.....	45
4.4.1 本文方法的有效性分析.....	45
4.4.2 本文方法参数权重配置.....	49
4.4.3 不同模块对本文方法的贡献.....	51
4.5 本章小结.....	53
第五章 面向在线评测系统的测试数据生成与评估工具原型设计	55
5.1 系统概述.....	55
5.2 系统架构设计.....	56
5.3 核心模块设计.....	57
5.3.1 测试数据评估.....	57
5.3.2 测试数据生成.....	59
5.4 系统实现.....	60
5.4.1 多维评估界面.....	61
5.4.2 测试生成界面.....	63
5.5 本章小结.....	66
第六章 结论与展望	67
6.1 研究工作总结.....	67
6.2 展望.....	68
参考文献.....	69

第一章 绪论

1.1 课题研究背景和意义

随着计算机领域的发展,编程能力在各行业中的重要性日益凸显。程序设计课程作为计算机科学教育的核心,已成为培养学生编程能力的关键环节。为了提升学生的编程水平,该课程通常需要大量的实践训练。然而,由于编程经验的缺乏,初学者所编写的程序常包含较多缺陷,进而增加了调试过程的难度。随着学生数量的增加,而教师资源相对有限,传统的人工批改与个别指导已难以满足需求。因此,如何为学生提供及时、有效的反馈,提高学习效率,成为了编程教育亟待解决的核心问题。

在线评测系统(Online Judge System, OJ)是一种能够自动评判程序并反馈执行结果的系统^[1]。OJ 系统允许教师在平台上上传编程题目及其对应的测试数据。学生提交程序后,系统会自动运行程序并检测其是否在规定的时间内、内存限制内通过所有测试数据,并给出评测结果。这种自动化评测机制不仅帮助学生快速验证自己的编程思路 and 实现,及时发现并修正错误,也显著减轻了教师的负担,使他们无需耗费大量时间在批改作业和验证程序正确性上^[2,3]。凭借其高效性和便利性, OJ 系统已被广泛应用于编程教学^[4,5]、算法竞赛^[6]和企业招聘^[1]等领域。

在 OJ 系统中,测试数据是评判程序正确性的核心要素。测试数据的质量直接影响评测结果的准确性与系统的公平性。一个合理且全面的测试数据集可以帮助学生全面检查程序的健壮性,确保程序在各种边界条件下均能正常运行。尤其在程序设计课程中,测试数据不仅用于验证学生程序的正确性,还能够揭示其代码中的潜在问题,帮助学生优化编程思路。因此,测试数据的质量对 OJ 系统的评测效果至关重要。

然而,尽管测试数据在 OJ 系统中扮演着重要角色,现有研究却集中在 OJ 系统的教育应用中^[6,7],缺乏关于测试数据有效性的研究。目前 OJ 系统在测试数据的设计与质量评估方面尚显不足。现有的测试数据通常依赖人工设计,这不仅耗费大量时间和精力,还容易受到设计者个人经验的局限,从而导致测试数据的全面性和合理性不足。人工设计的测试数据往往难以涵盖所有可能的程序错误,尤其是在应对复杂边界条件和特殊输入时,容易忽略某些细节。此外,随着 OJ 平台题目数量的增长,测试数据的维护与更新愈发繁琐,人工设计的效率和质量呈下降趋势。

因此,如何高效评估现有测试数据质量以及自动生成高质量的测试数据,已成为提升 OJ 系统质量的重要方向。通过引入科学化、系统化的评估和生成方法,不仅能够增强测试数据的全面性和准确性,还能显著降低维护成本,从而提升 OJ 系统的整体运行效率。这一问题的解决将为 OJ 系统的广泛应用带来深远影响。

1.2 研究现状

1.2.1 测试数据评估

软件测试作为保障软件质量的核心实践手段，其有效性验证始终面临根本性挑战。计算机科学家 Dijkstra 于 1972 年提出的经典论断“程序测试只能证明存在缺陷，而无法证明其不存在”深刻揭示了测试活动的局限性^[8]。在此理论框架下，如何科学评估测试数据的缺陷检测能力，构建有效的质量评价体系，成为软件测试领域持续关注的核心问题。

测试充分性标准作为评估测试质量的理论基础，其研究可追溯至软件测试理论的早期发展阶段。1988 年，Weyuker^[9]等人从公理化视角探讨了测试数据充分性标准，提出语句覆盖属性等核心公理，为评估指标的形式化验证奠定了理论基础。1997 年，Zhu^[10]等人系统梳理了过去二十年的研究成果，提出了测试充分性准则的分类框架，并从理论、方法和实践角度进行了深入分析。当前主流的测试评估标准主要围绕代码覆盖率和变异测试两大维度展开^[10,11]。

代码覆盖率由行覆盖率与分支覆盖率组成。在软件测试实践中，测试人员期望生成测试数据可以执行程序中的全部语句。行覆盖率以执行语句数与总语句数的百分比来量化这一目标，其数值直观反映语句层面的测试完整性。值得注意的是，源代码中的条件语句（如 if 语句）会生成两个独立执行路径，完善的测试需要覆盖所有可能的分支路径。分支覆盖率通过统计已覆盖分支占全部分支的比例，进一步评估测试数据对程序控制流的覆盖质量。然而，语句覆盖并不等同于错误检测，测试数据可能覆盖了某行代码，但是却忽略了代码中的错误^[12]。因此，现有的研究尝试改进代码覆盖率，例如将代码覆盖率与其它特征综合，使得该指标更加可信^[13]。

变异测试由 DeMillo^[14]等人提出，其核心思想是通过模拟真实错误场景来评估测试数据的错误检测能力。该方法包含三个关键步骤：首先向原始程序注入预定义的语法变异（如运算符替换、语句删除等），生成称为变异体的错误程序；随后，使用测试数据来执行这些生成的变异体；最终通过比对变异体与原始程序的输出差异判定测试有效性。如果变异体和原始程序在至少一个测试数据上产生不同的输出，则认为测试数据检测到了错误，称该变异体被杀死。变异得分即为被杀死变异体数量与总变异体数量的比值，该指标能更真实地反映测试数据的错误检测能力。然而，尽管这种方法具有更高的可靠性，但是其计算成本较高^[15]。为提高变异测试的实用价值，现有研究在多个维度进行技术优化，如面向变异体的优化方法^[16,17,18]、面向测试数据的优化方法^[19,20]和面向变异执行的优化方法^[21]。

尽管上述两类评估方法及其改进版本在工业领域中已取得了显著成果，但其在 OJ 系统中的应用仍存在局限性。具体而言，现有方法在评估维度上尚不全面，难以有效

衡量 OJ 系统中学生与教师普遍关注的测试规模等关键指标，也缺乏对测试数据中可能存在的漏报问题的检测能力。因此，如何全面、高效地评估 OJ 系统中人工测试数据的质量是一个重要的研究方向。

1.2.2 测试数据生成

软件测试在软件开发过程中扮演着至关重要的角色，其效率与完备性直接决定最终交付产品的可靠程度。然而，软件测试也常常是一个高成本、高资源消耗的环节。研究表明，软件测试通常占据整个软件开发成本的 30%至 50%^[22,23,24]。并且，随着系统规模的扩大，测试的复杂度和成本也会急剧增加。因此，如何提升测试效率、降低测试成本，成为了软件工程领域的一个重要研究方向。

在众多测试活动中，测试数据的生成是最为关键且具有挑战性的任务之一^[22]。测试数据的质量直接影响其错误检测能力与代码覆盖水平，然而人工构造测试数据集的过程存在效率低下和错误率高等局限性。特别是在针对复杂软件系统进行测试时，人工设计的测试数据在边界条件覆盖率和异常场景完备性方面难以充分满足测试目标的要求。因此，为提升测试的全面性与效率，研究人员提出了多种自动化测试数据生成技术，旨在借助算法手段替代人工设计过程，从而实现测试数据生成高效化。

1975 年，King^[25]提出了符号执行技术，它通过使用符号而非具体的变量值作为程序的输入，系统性地探索程序可能的执行路径。符号执行的一个重要优势在于能够发现代码中的潜在错误和边界条件。因此，研究人员基于符号执行提出了多种测试数据生成技术^[26,27,28,29]。然而，符号执行在实际应用中也面临着一些挑战，尤其是路径爆炸^[30,31]、路径发散^[32,33]以及复杂约束^[34,35]。

基于搜索算法的测试数据生成技术是一种流行的测试数据生成方法^[36,37]。这类方法将测试数据生成建模为组合优化问题，通过设计适应度函数量化测试目标，结合遗传算法、粒子群优化等智能搜索策略，在解空间内高效寻找最优测试数据集。现有研究不断提出新的搜索算法，旨在较短时间内生成更加高效、均衡的测试数据^[38,39]。

近年来，大语言模型（Large Language Model, LLM）的突破性发展为测试数据的生成带来新的方法。以 ChatGPT 为代表的 LLM 展现出了卓越的语义理解与逻辑推理能力，研究人员开始探索 LLM 在软件工程领域中的应用，其中便包含测试数据自动生成。然而，单纯依赖 LLM 来生成高质量的测试数据仍然面临诸多挑战^[40,41]。例如，模型的输出内容可能不符合实际的测试约束条件，或者生成的测试代码无法编译。因此，研究人员尝试通过提示词工程来改进模型的生成结果^[42]，或是将 LLM 与其他技术结合，进一步提升测试数据生成的质量和效果^[40,43]。

现有方法在实验中取得了较为理想的效果，其生成的测试数据在错误检测率和代码覆盖率方面表现优异。然而，当前研究主要集中于工业软件中的测试套件生成，针

对 OJ 系统中算法题目测试数据生成的研究相对较少。在 OJ 系统中，部分题目对测试数据规模有较高要求，需包含数万甚至百万级别的测试元素，这对现有生成方法提出了较大挑战。此外，现有方法的生成目标往往与实际需求不完全一致，教师与学术更关注的是以最小规模覆盖所有错误程序的测试数据集。因此，研究面向 OJ 系统的测试数据生成技术变得尤为重要。

1.3 课题主要研究内容

针对现有测试数据有效性不足导致出现漏报现象的问题，本课题从测试数据评估和生成两个方面进行了深入研究。首先，本课题从多个维度对 OJ 系统中现有测试数据集进行了全面评估，并针对现有方法测试冗余、效率较低的问题，提出了一种基于聚类 and 差分测试的假阳性程序检测方法。该方法利用学生程序的动态特征进行聚类，避免重复检测，提高测试效率。其次，本课题提出了基于大语言模型和多目标优化算法的测试数据生成方法，生成兼顾错误检测能力和测试规模的高质量测试数据集。最后，本课题基于研究内容设计并实现了系统原型，以支持编程教学实践。本课题的主要研究内容包括：

1. 本课题在 BuctOJ 和 AtCoder 两大流行的在线评测系统中收集了大规模学生程序数据集，并使用现有研究中最常用的代码覆盖率与变异得分对现有测试数据进行评估。针对 OJ 系统中测试数据在辅助学生复现错误和修复程序方面的独特作用，本文提出了测试规模作为衡量指标，进一步丰富了评估维度。

2. 本文提出了一种基于聚类 and 差分测试的假阳性程序检测方法。基于 OJ 系统中算法题目解法有限，学生程序高度相似的特点，本文提取了学生程序的动态特征，并通过聚类将海量程序划分为少量代表性族群，从而显著降低后续差分测试的计算开销。其次，本文基于题目信息和学生程序的静态特征，生成了针对边界条件、分支逻辑和数据上限等关键场景的测试数据集，通过差分测试检测假阳性程序。

3. 本文面向 OJ 系统提出了一种基于大语言模型与多目标优化算法的测试数据生成方法。该方法利用了 LLM 的语义理解能力与代码生成能力，通过提示词工程生成多样化的测试数据集。随后以此为基础，采用多目标优化算法迭代优化，以错误检测率、代码覆盖路径和测试规模的加权和作为适应度函数，生成兼顾错误检测能力与测试规模的高质量测试数据。针对代码覆盖率指标忽视稀有路径的局限性，本文提出了一种基于覆盖频次的动态权重调整策略，提升测试数据对代码路径的覆盖多样性。

4. 本文基于 BuctOJ 和 AtCoder 平台收集的 33205 份学生程序开展了实验研究。结果显示，现有测试数据的平均行覆盖率与分支覆盖率为 81.71% 和 79.35%，平均变异得分为 0.81，平均测试元素数高达 76692，表明测试数据质量仍有较大的提升空间。在假阳性程序检测方面，相比于 DT 方法，本文方法仅使用其 50.39% 的时间开销，实

现了更高的 Recall 值和 F-measure。在测试数据生成方面,本方法的平均错误检测率达到了 97.62%,相比于 GA 方法和现有测试数据提升了 6.77%和 6.07%,行覆盖率分别提升了 3.6%和 2.96%,分支覆盖率分别提升了 4.33%和 3.96%,测试规模分别减少了 5.05 倍和 6.64 倍。此外,相较于 AID 方法,本文方法在假阳性程序检测率上提升了 19.44%。

5. 基于上述研究成果,本文设计并实现了测试数据评估和生成工具——TestEG。该工具集成了本文提出的测试数据质量评估方法和测试数据自动生成技术,支持管理员深入分析现有测试数据的缺陷并生成高质量测试数据,为 OJ 系统的优化与编程教学提供了实用支持。

1.4 本文组织结构

本文分为六章,结构如下:

第一章 本章介绍了课题的研究背景及测试数据的评估与生成领域的研究现状。从编程教育快速发展的现实需求出发,分析了 OJ 系统中测试数据的关键性作用,并指出了目前 OJ 系统测试数据质量模糊的问题,进而说明了高效评估测试数据质量以及自动化生成测试数据的必要性。

第二章 本章系统介绍了课题研究的技术背景,包括 OJ 系统的工作原理及其潜在风险,以及现有研究中常用的多目标优化算法。随后,本章介绍了与本文相关的研究工作,涵盖测试数据评估方法和自动生成技术的最新进展。

第三章 本章详细介绍了多维测试数据评估方法。本章分析了 OJ 系统中算法题目的特性及测试数据可能存在的缺陷,随后从代码覆盖率、变异得分和测试规模三个维度对现有测试数据质量进行评估。针对 OJ 场景下学生程序的特点,本文提出了一种基于聚类 and 差分测试的假阳性程序检测方法。在 BuctOJ 和 AtCoder 数据集上开展实验,详细分析了现有测试数据的质量,并通过对比现有方法验证了本文方法的有效性。

第四章 本章介绍了基于大语言模型和多目标优化算法的测试数据生成方法。本章首先分析了 LLM 和遗传算法的优缺点,并基于两者优势提出了一种测试数据两阶段生成方法。随后详细介绍了 LLM 的生成策略及多目标优化算法的迭代过程。最后,在第三章所构建的数据集上开展的实验结果表明,本文方法生成的测试数据在提升错误检测能力的同时显著降低了测试规模。

第五章 本章介绍了基于研究成果设计的测试数据评估与生成工具——TestEG 的设计与实现过程。该工具实现了多维测试数据质量评估和自动化生成功能,为编程教学和 OJ 系统优化提供了实用支持。

第六章 本章总结了本文的研究内容,并指出了面向 OJ 系统测试数据的进一步研究方向。

第二章 背景和相关工作

2.1 在线评测系统

在线评测系统（Online Judge System, OJ）最早应用于 ACM 国际大学生程序设计竞赛（ICPC），其核心功能是自动验证参赛选手提交的程序是否正确，并检查程序在运行过程中是否超出资源限制，例如时间和内存^[1]。此外，OJ 系统还可以根据参赛选手解决题目的数量与解题时间生成实时排名，极大地提升了竞赛的组织效率和公平性。基于 OJ 系统的高效性和便利性，它逐渐被广泛应用于各类编程竞赛中。

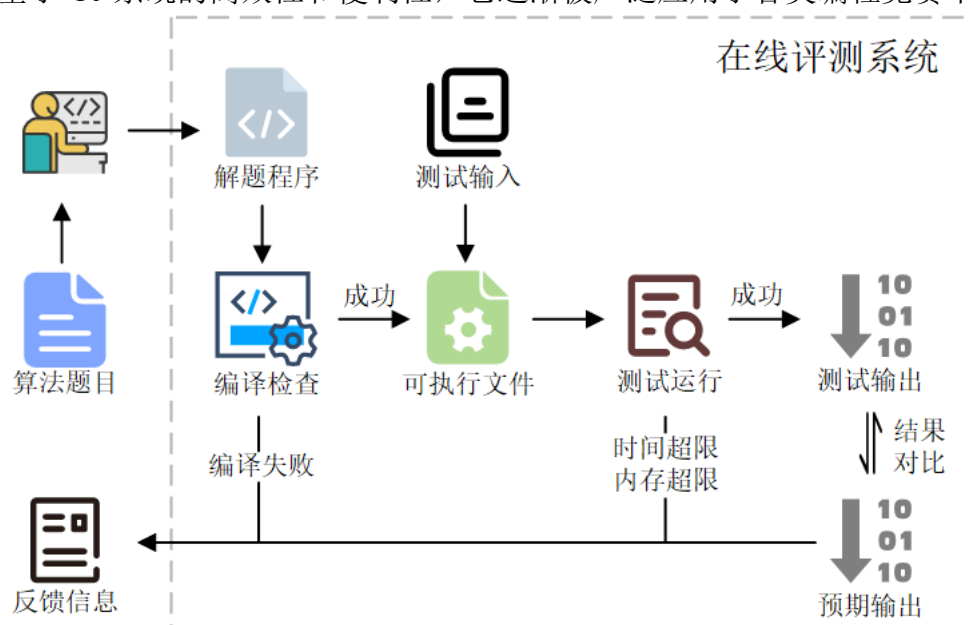


图 2-1 OJ 系统流程图

Fig.2-1 Online judge system workflow

图 2-1 为 OJ 系统的工作流程图，其工作原理如下：

1. 学生首先阅读题目描述，编写解题程序并将代码提交至 OJ 系统。
2. OJ 系统接收代码后，尝试将其编译为可执行文件。如果编译失败，系统会直接返回编译错误信息。
3. 编译成功后，OJ 系统会调用内置的判题机对可执行文件进行测试。判题机会依次输入一组预先设计好的测试数据，并将程序运行结果与预期输出进行比对。在此过程中，如果出现运行错误、答案错误、时间超限或内存超限等情况，系统将返回对应的错误信息。
4. 如果程序在所有测试数据上均通过，则返回“通过”的结果，并显示执行用时与内存消耗等指标。整个评测过程通常在几秒内完成，显著提高了程序评测的效率。

目前,随着计算机专业和互联网行业的兴起,OJ 系统的应用场景逐步扩展,除了在竞赛中发挥关键作用外,还被用于学生的日常编程训练以及企业的人才选拔。许多高校利用 OJ 系统辅助程序设计课程教学,帮助学生在实际练习中巩固理论知识;而企业则将 OJ 系统作为笔试环节的重要工具,通过自动化评测快速筛选出具备优秀编程能力的求职者。

OJ 系统在发展中也逐渐完善,其题库中涵盖了大量具有不同难度和类型的算法题目,为学生和开发者提供了丰富的练习资源。学生通过在 OJ 系统中刷题,能够快速掌握算法设计与问题求解能力,并不断提高编程技巧。这不仅有助于巩固课程知识,还可以为毕业生参加企业技术面试做好充分准备。此外,OJ 系统还支持竞赛模式,教师和企业可以基于平台组织实时竞赛,评估参与者的编程能力与问题解决能力。

总体而言,OJ 系统在编程竞赛、教学辅助、学生训练以及企业招聘等领域的广泛应用,极大地推动了编程教育的发展,也为培养高质量的计算机人才提供了有力支持。

2.2 多目标优化算法

多目标优化算法旨在同时优化多个相互独立甚至可能冲突的目标,从而为复杂问题提供一组均衡的解^[44]。与传统的单目标优化算法相比,多目标优化算法不仅关注单一目标的最优性,更致力于在多个目标之间寻找一种平衡^[45]。由于多个目标往往彼此制约,通常难以得到使得所有目标同时达到全局最优的结果。因此,在多目标优化问题中,解的优劣通常不能通过单一指标来衡量,而是需要通过帕累托前沿(Pareto Front)来描述。在帕累托前沿上,每个解在某些目标上已达到最优水平,而对其他目标的让步则保持在可接受范围内。算法的目标是探索解空间,尽可能获得一组多样性高且分布均匀的帕累托最优解,以便决策者根据实际需求选择最合适的方案。

本文选取了当前研究中具有代表性的两种多目标优化算法进行介绍,这些算法因其解决不同场景下多目标问题的出色表现而被广泛研究、使用和对比^[46,47,48]。

2.2.1 非支配排序遗传算法

非支配排序遗传算法(Non-dominated Sorting Genetic Algorithm II, NSGA-II)^[49]是一种广泛应用的多目标优化算法,基于帕累托最优原理实现对解的优化排序。NSGA-II 通过非支配排序将种群中的个体划分为多个等级,优先保留较低等级的个体,以逐步逼近帕累托前沿。算法的核心是以种群规模为限制,在每一代迭代中优先选取非支配解集作为候选解。

当候选解集的大小超过种群限制时,NSGA-II 引入了拥挤度计算方法。通过评估每个解在目标空间中的分布密度,算法优先保留拥挤度较低(即分布更均匀)的解,

从而提高帕累托解集的多样性。NSGA-II 高效的非支配排序与拥挤度排序机制，使其在处理复杂多目标问题时表现优异。

2.2.2 强度帕累托进化算法

强度帕累托进化算法（Strength Pareto Evolutionary Algorithm II, SPEA-II）^[50]是另一种基于帕累托最优的多目标优化算法，但其独特之处在于适应度赋值方式。SPEA-II 为种群中的每个个体分配适应度值，该值综合考虑个体的帕累托支配关系与其在目标空间中的分布密度。

SPEA-II 在每次迭代中维护一个存储非支配解的外部档案，用于记录种群中当前的优秀解。当非支配解集的规模超过档案容量时，算法会基于适应度值对解集进行截断，优先保留分布均匀、质量较高的解。SPEA-II 的适应度计算与档案管理机制为其提供了强大的解探索与保存能力，使其在多目标优化问题中表现出色。

2.3 测试数据评估方法

测试数据作为软件测试中的核心组成部分，其质量直接影响着测试的效果，以及软件最终的出品质量。因此，关于测试数据的评估是一个长期研究的课题。从 20 世纪 70 年代便开始了关于测试数据评估问题的研究，当时的研究主要集中在代码覆盖率和变异得分上。而随着计算机的发展，软件的规模和访问量都日益庞大，测试数据的规模和复杂度也日益提高。这两项指标都不足以完全衡量测试数据的质量，因此，研究人员在这两项指标的基础上展开深入研究。

在代码覆盖率方面，Malaiya^[51]提出了一种对数-指数（LE）模型，用于关联测试覆盖率与缺陷覆盖率及软件可靠性。Alireza^[52]提出了一种基于覆盖率的指标——语句频率覆盖率。他为每行代码语句分配了一个连续的值，这个值与改行代码在测试数据中的执行次数成正比。实验结果表明，语句频率覆盖率优于行覆盖率与分支覆盖率。并且与变异得分的相关性更高。Khashayar^[13]提出了一种新的测试覆盖率度量指标——程序状态覆盖（PSC）。传统的代码覆盖率仅关注代码行是否被执行，而忽略了同一行在不同程序状态下可能存在的不同行为。PSC 通过记录每个代码行执行时所处的程序状态（由程序计数器值和所有变量值定义），量化测试套件对程序状态的覆盖程度，从而更准确地评估测试有效性。此外，工业界也开发了 Agitar^[53]、Dynamic^[54]等多种覆盖率工具，通过动态插桩生成测试报告。

在变异测试方面，Delgado-Pérez^[55]对 c++ 的类级变异操作符进行了评估。基于这些结果，他们提出了一个 C++ 变异工具 MuCPP，它通过使用 Clang API 遍历每个翻译单元的抽象语法树来生成变异体。为了缩减变异测试的执行成本，Gopinath^[56]等人在

大型开源程序上进行了实验。他们发现随机选择的少量恒定数量的变异体能够提供与使用所有变异体时统计上相似的结果。并且该样本与程序大小和变异体直接的相似性无关。由于真实世界中，一个程序中通常存在多个 bug，而一阶变异体仅会引入一个错误，无法模拟这种真实情况。Jia^[57]等人提出了高阶变异测试，通过向程序中植入多处人工错误来生成高阶变异体，进而更好的衡量测试数据在真实场景下的错误检测能力。Papadakis 和 Malevris^[58]等人发现二阶变异体可以减少 80%至 90%的等价变异体，而测试的有效性损失约为 10%或更少。Kintis^[59]等人的研究也给出了类似的结论，他们发现二阶变异体的测试效率明显高于一阶变异体。随着深度学习的发展，研究人员也尝试将其应用到变异测试中。如 Tufano^[60]提出了 DeepMutation，这是一个基于 RNN 的深度学习模型，该模型可以生成更接近真实错误的变异体，并通过自动化工具链实现变异体的生成、注入和测试。

尽管上述方法在软件测试领域已取得显著成效，但在 OJ 系统的应用场景中仍存在显著局限性。例如，在 OJ 系统中可能存在漏判的情况，即一些包含错误的程序被判定为了“正确程序”。在这种情况下，虽然测试数据虽具有高覆盖率与变异得分，但是它们仍未能检测出程序中的潜在错误。针对这一核心问题，Liu^[61]等人创新性地提出“假阳性程序”量化指标，通过统计漏判程序数量来评估测试集质量。

2.4 测试数据生成方法

测试数据自动生成是软件工程中的一个重要研究方向，旨在通过自动化手段生成有效的测试数据，以提高软件测试的效率和质量。自动化生成测试数据能够节省大量人力成本，确保测试的全面性和系统的鲁棒性。随着研究的不断深入，测试数据自动生成技术经历了多个阶段的发展。从最初的基于符号的生成方法，到后来的基于搜索的生成技术，再到最近基于 LLM 的自动化生成方法，测试数据生成的技术手段和应用场景日益丰富。

2.4.1 基于符号执行的测试数据生成

符号执行（Symbolic Execution）是一种静态程序分析技术，其主要通过将程序中的常量值替换为符号变量，并在程序执行过程中跟踪符号值的变化，来生成测试数据。通过符号执行，能够系统地探索程序的所有可能执行路径，并识别潜在的错误和漏洞。符号执行技术的核心优势在于其能够提供高覆盖率的测试数据，特别是对于复杂的程序结构和边界条件。Castro^[62]等人使用符号执行生成测试数据以在开发环境中重现在用户端出现的故障，并且该行为不会泄露用户的隐私。Santelices^[63]等人借助符号执行

来揭示同一软件在不同版本下的行为差异。Majumdar^[64]使用符号执行生成测试数据，这些测试数据的轻微扰动会引起程序输出的显著差异。

2.4.2 基于搜索算法的测试数据生成

基于搜索算法的测试数据生成方法将测试生成问题建模为优化问题，利用启发式搜索算法在输入空间中寻找满足特定测试目标的测试数据。该方法的核心在于定义适应度函数，通过遗传算法（GA）、粒子群优化（PSO）等元启发式算法迭代优化测试覆盖率、缺陷检测率等指标。目前已有许多关于 SBTCG 的研究，例如：Avila-George^[38]等人提出了一种基于 SA 算法的元启发式方法，通过两个唯一的邻域函数降低了求解成本，最终获得了比基准方法更好的结果。Srivastava^[65]等人研究了应用遗传算法和蚁群算法生成资源请求算法测试数据的可行性，他们的结果证明了这两种技术在软件测试应用程序中自动生成测试数据的可行性。Hashim^[66]等人提出了一种新的优化算法，称为蜜獾算法（HBA），该算法模仿蜜獾的觅食行为，以平衡探索和利用能力。他们在基准函数和实际工程设计问题上对 HBA 进行了测试，实验结果表明 HBA 可以有效地解决复杂搜索空间的问题。

2.4.3 基于大语言模型的测试数据生成

基于 LLM 的测试生成技术利用 LLM 强大的自然语言理解能力和代码生成能力，通过提示词工程或对 LLM 进行微调，进而生成具有更高覆盖或更易触发程序异常行为（如边界条件错误、逻辑错误等）的测试数据。Lemieux^[67]等人提出了 CODAMOSA，该方法将 LLM 与搜索算法相结合，当搜索过程中测试数据的覆盖率陷入停滞时，CODAMOSA 要求 LLM 为未覆盖的功能区域生成测试数据，最终实现了更高的代码覆盖率。Deng^[68]等人提出了 TitanFuzz，他们使用 LLM（即 Codex）来生成有效的深度学习程序来测试深度学习库。在 PyTorch 和 TensorFlow 上的实验结果表明，TitanFuzz 可以实现比目前最先进的模糊测试工具更高的代码覆盖率。Li^[69]等人提出了 Differential Prompting，该方法利用了 ChatGPT 在理解程序意图时，对程序内部的实现细节并不敏感的特性。通过程序意图推断、程序生成和差分测试三个步骤，显著提升了生成诱导失败的测试数据的成功率。该方法在 QuixBugs 和 Codeforces 的实验中分别达到了 75.0% 和 66.7% 的成功率，远超基准方法。

2.5 本章小结

本章系统梳理了 OJ 系统、多目标优化算法、测试数据评估方法以及测试数据自

动生成技术的研究背景与发展现状。首先，介绍了 OJ 系统的起源、工作原理及其在编程竞赛、教育训练和企业招聘中的广泛应用，为后续指出其测试数据质量问题所带来的潜在威胁进行铺垫。其次，介绍了多目标优化算法的基本原理，重点分析了 NSGA-II 和 SPEA-II 两大代表性算法的特点。此外，本章回顾了测试数据评估领域的经典指标——代码覆盖率和变异得分及其改进方法，指出这些方法在 OJ 系统场景下难以全面衡量测试数据质量的局限性。最后，介绍了测试数据自动生成技术的发展历程，包括基于符号执行、搜索算法和 LLM 的方法。

第三章 基于聚类与差分测试的测试数据评估方法研究

本章对 OJ 系统中人工设计的测试数据进行了多维评估,提出了一种基于动态特征的聚类算法和基于静态特征的差分测试方法。该方法旨在对学生程序进行约简,并生成更易触发错误的测试数据,从而在保证假阳性程序检测精度的同时,大幅缩减执行成本。

3.1 研究动机

OJ 系统被广泛用于编程教育、算法竞赛、企业招聘等场景中,其判别机制完全依赖管理员上传的测试数据。具体来说,管理员通常提供题目描述及相应的测试数据集, OJ 系统执行学生提交的程序,并将其输出与预设的标准答案进行比对,以判定程序的正确性。如果结果不一致,则返回“答案错误”的反馈。因此,测试数据的质量对于 OJ 系统的准确性至关重要。

目前,关于测试数据质量的研究已在工业应用场景中取得诸多成果,但针对 OJ 系统的研究仍较为薄弱^[61]。然而, OJ 系统中的测试数据集在有效性方面仍面临诸多挑战,尤其是在算法题目中,主要表现为以下特性:

类型繁多: OJ 系统中的算法题目涵盖广泛,如树、图、动态规划、搜索等,题目类型多样,测试数据需全面覆盖不同的解法和边界情况。然而,即便是经验丰富的出题人,也难以确保测试数据完全涵盖所有边界情况,人工设计的测试数据往往存在覆盖不足的漏洞。

输入数据规模庞大: 大多数算法题目的输入规模上限较高,通常可达数万行数据。面对如此庞大的数据规模,手动编写测试数据几乎不可行,管理员通常通过编写脚本随机生成测试数据。尽管随机生成的方法能够快速产生大规模测试数据,但其在覆盖范围和有效性方面存在局限性,可能无法精准捕捉程序中存在的缺陷。

这些挑战直接导致 OJ 系统中漏报现象的发生。漏报指学生提交的程序存在缺陷,但由于测试数据集的不完备,系统错误地判定其为“正确程序”。此类程序被称为“假阳性程序”。漏报不仅影响系统对程序正确性的判定,还会对学生的学习效果以及题目设计者的意图产生负面影响。

如下所示为 BuctOJ 系统中的题目 2130 的一份学生程序,该程序被 OJ 系统判定为正确。然而,该题目要求输出求和大于 0 的最大连续子序列的长度及其起止位置下标,学生程序的第 17 行定义了初始值为-1 的起止位置变量 x 和 y 。当输入序列仅包含一个正整数时,程序由于没有适当更新 x 和 y 的值,最终输出了未被修改的初始值。由于人工在设计测试数据时仅考虑了包含一个负数的序列,对问题边界情况的覆盖不

完整，导致该程序被错误地判定为正确，从而产生漏报。

假阳性程序示例：

```

1. #include <bits/stdc++.h>
2. using namespace std;
3. int main()
4. {
5.     int n;
6.     while(cin>>n)
7.     {
8.         int arr[100000+5],flag=0;
9.         for(int i=0;i<n;i++)
10.        {
11.            cin>>arr[i];
12.            if(arr[i]>0)
13.                flag=1;
14.        }
15.        if(flag)
16.        {
17.            int sm=0, mx=arr[0], x=-1, y=-1, ind=0;
18.            for(int i=0;i<n;i++)
19.            {
20.                sm+=arr[i];
21.                if(sm<0)
22.                {
23.                    sm=0; ind=i+1;
24.                }
25.                if(mx<sm)
26.                {
27.                    mx=sm; x=ind; y=i;
28.                }
29.            }
30.            cout<<mx<<" "<<x<<" "<<y<<endl;
31.        }
32.        else
33.            cout<<0<<" "<<-1<<" "<<-1<<endl;
34.        flag=0;
35.    }
36.    return 0;
37. }

```

与漏报相对的是误报，即学生提交的程序完全正确、符合题目要求，但却被错误判定为“答案错误”。然而，在 OJ 系统中，误报的情况极为罕见。这是因为每道题目在发布前都会提供标准解法（由出题人编写），并且测试数据集通常经过标准解法验证和人工审核的双重检验，以确保其符合题目要求的正确性。

针对上述问题，Liu^[61]等人提出了一种基于差分测试（Differential Testing, DT）的假阳性程序检测方法来调研 OJ 系统中的漏报情况。然而，该方法需要生成并执行大量的测试数据，检测效率过低。OJ 系统在长期运行中会积累大量的学生程序，DT 方法的执行开销将随时间增长而显著上升。

此外，OJ 系统在高校中被广泛用于编程教学，帮助学生验证程序，辅助其发现

程序中的错误。然而，当前测试数据的设计更多侧重于发现错误，导致许多算法题目的测试数据规模异常庞大。虽然大规模测试数据通常具备更高的代码覆盖率和变异得分，但当学生提交的程序存在错误时，这些用例往往只能给出判定结果，难以提供进一步的有效反馈以帮助学生修改程序。面对海量输入数据，学生往往难以利用测试数据来快速定位程序的错误位置。

因此，亟需研究更加高效且全面的验证方法，以在合理时间内评估测试数据集的有效性。

3.2 在线评测系统测试数据评估框架

传统的单维度或双维度测试数据评估方法在 OJ 系统中难以全面衡量测试数据的优劣，往往存在评估视角片面、指标失衡等问题。本文构建了包含代码覆盖、错误检测与测试规模的三维评估框架，通过多维度量化分析实现测试数据质量的立体化评估。该框架不仅考虑了技术层面的有效性指标，同时兼顾教育场景下的辅助需求。

3.2.1 代码覆盖

代码覆盖是测试数据评估中最流行也是最核心的考量之一。测试数据仅能验证其执行路径覆盖的代码行为，未被测试覆盖的代码区域则可能隐藏潜在错误。因此，越全面的代码覆盖意味着更充分的测试。本文采用了混合插桩技术实现，结合编译时插桩与运行时追踪，计算程序的代码覆盖程度。具体方法如下所示。

（1）程序插桩编译

使用 GCC 编译器启用覆盖率分析功能，通过编译选项 `-fprofile-arcs -ftest-coverage` 对被测程序进行插桩改造。该过程在 LLVM 中间表示层自动插入探针代码，形成具有覆盖率采集能力的可执行文件。其插桩原理可表示为：

$$P' = \Phi(P, I) \quad (3-1)$$

其中 P 为原始程序， I 为插桩指令集， Φ 表示插桩变换过程。

（2）测试数据动态执行

在测试环境中执行插桩后的程序，输入测试数据集 $T=\{t_1, t_2, \dots, t_m\}$ ，通过重定向机制获取每个测试数据的执行轨迹。在程序执行过程中产生 `.gcda` 二进制文件，记录分支跳转次数、代码行命中次数等运行时信息。

（3）覆盖率数据采集

调用 `gcov` 工具解析插桩数据，生成可读的 `gcov` 报告文件。该文件采用标记化格

式记录每行代码的执行状态。其中，未覆盖行会被标记为“#####”，已覆盖行会显示执行次数，分支点则会标注“branch”标签以及跳转次数。数据解析算法通过正则表达式匹配关键标记。

(4) 覆盖率指标计算

基于解析结果计算两类核心指标：行覆盖率与分支覆盖率，计算公式如 3.3.3 节所示。

3.2.2 错误检测

代码覆盖反映的是测试数据对程序的潜在错误检测范围，无法完全体现其错误检测能力。变异测试则反映了测试数据对程序变化的敏感程度，可以更有效的衡量出测试数据的错误检测能力。其核心思想是通过对程序进行变异操作，模拟潜在的编程错误，并通过测试数据的执行情况来衡量其识别和捕捉错误的能力。具体的方法如下所示：

表 3-1 变异算子
Table 3-1 Mutation operator

变异算子	描述	例子
CRCR	常数替换	$a=b + *p \rightarrow a=0 + *p$
OAAN	算术运算符变异	$a + b \rightarrow a * b$
OAAA	算术赋值运算符变异	$a += b \rightarrow a -= b$
OCNG	逻辑谓词否定	$\text{if}(a) \rightarrow \text{if}(!a)$
OIDO	自增/自减变异	$++a \rightarrow a++$
OLLN	逻辑运算符变异	$a \&\& b \rightarrow a \parallel b$
OLNG	逻辑否定	$a \&\& b \rightarrow !(a \&\& b)$
ORRN	关系运算符变异	$a < b \rightarrow a \leq b$
OBBA	位赋值变异	$a \&= b \rightarrow a = b$
OBBN	位运算符变异	$a \& b \rightarrow a b$
OCOR	类型运算符替换	$\text{int } a \rightarrow \text{float } a$
SRSR	返回语句替换	$\text{return } 0 \rightarrow \text{return } 1$
VTWD	常数增减	$a = b \rightarrow a = b + 1$
VDTR	域变换	$c = a \rightarrow c = a * 0$
SSDL	语句删除	$a = 1 \rightarrow \langle \text{no-op} \rangle$

(1) 生成变异体

在进行变异测试时,首先对程序源代码进行小范围的修改,模拟常见的编程错误,这些修改称之为变异算子。例如,改变运算符的类型、修改常量值、改变条件判断等。本文使用的变异算子如表 3-1 所示。在原程序上使用变异算子后产生的错误程序称之为变异体。

(2) 执行变异体

在人工测试数据集 $T=\{t_1, t_2, \dots, t_m\}$ 上执行变异体并记录每个变异体的输出结果,随后将变异体的输出与原程序的输出进行对比。如果输出不同,则认为测试数据识别到了程序中的错误,称为变异体被“杀死”。如果输出相同,则测试数据未能发现程序中的错误,变异体存活。

(3) 计算变异得分

变异得分统计了测试数据杀死的变异体所占比例,计算公式如 3.3.2 节所示。

3.2.3 测试规模

代码覆盖率与变异得分分别从不同层面对测试数据进行了评估,其核心都在于测试数据的错误检测能力。对于工业软件而言,错误检测是测试数据的核心目标,甚至是唯一目标,因此测试数据的设计通常无需考虑其规模。然而,在 OJ 系统中,测试数据不仅用于评估程序的正确性,还承担着重要的教育功能。当学生收到“答案错误”的反馈时,他们往往希望通过失败测试数据来复现错误并修复程序。然而,过于复杂或庞大的测试数据会降低其可读性,使学生难以利用,无法理解错误原因。因此,在 OJ 场景下,测试数据的规模也是一个关键因素,需要在测试充分性与可理解性之间取得平衡。本文从以下三个角度对测试数据的测试规模进行衡量:

(1) 测试数据集

由于算法题目通常涉及复杂的算法逻辑及多个边界条件,单个测试数据往往无法充分验证学生程序在所有可能场景下的正确性。因此,OJ 系统中的算法题目通常由多个测试数据组成,形成完整的测试数据集,以覆盖不同的程序边界和异常情况,确保测试的全面性与健壮性。为评估不同题目的测试覆盖情况,本章节统计了数据集中每道算法题目的测试文件数量,并将其作为测试数据集的规模指标。

(2) 测试数据行数

在大多数算法题目中,输入数据通常由多行组成,每一行的数据可能代表问题的不同部分或特定的测试条件,其复杂度直接影响学生调试的难度及程序设计的思路。测试输入的行数越多,通常意味着题目输入规模更大,可能包含更多的测试场景。因此,本章节统计了每个测试数据的行数,并汇总为该题目的测试数据行数,以衡量测试输入的复杂度。

(3) 测试元素数

在算法题目中，每一行输入数据通常由多个整数或字符组成，学生需要正确解析这些元素，并基于其逻辑关系实现程序。测试元素的数量不仅影响程序的计算复杂度，也可能影响学生对题目要求的理解及实现策略的选择。为了量化输入规模，本章节基于空格分隔符统计每一行的测试元素数量。

3.3 基于程序动态和静态特征的假阳性程序检测方法

尽管 3.2 节从不同维度上分析了 OJ 系统中现有测试数据的有效性，但这些方法无法反应测试数据是否完成了其核心目标——检测出程序中的全部错误。Liu^[61]等人创新性的提出了“假阳性程序”指标，通过 DT 方法检测所有的“正确程序”中是否存在未被检测出的错误。这种指标可以更加直观的反应出 OJ 系统中测试数据的完备性。然而该方法的执行成本极高，并且存在资源浪费。为了提高假阳性程序检测的效率，本文提出了一种基于程序动态和静态特征的假阳性程序检测方法。该方法主要包含两个步骤：基于动态特征的学生程序聚类 and 基于静态特征的差分测试。方法流程如图 3-1 所示。

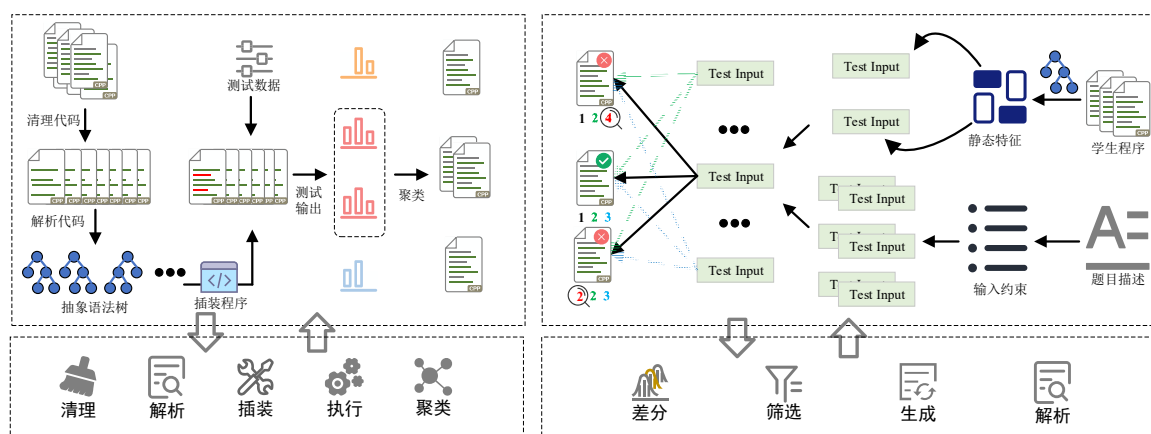


图 3-1 DFC-SFDT 方法流程图

Fig.3-1 Flowchart of the DFC-SFDT approach

3.3.1 基于动态特征的学生程序聚类

与传统工业软件相比，OJ 系统中的题目描述通常较为固定，不涉及需求变更或软件更新。因此，此类题目的解法具有确定性和有限性。尽管在长期运行中，某道题目可能积累了上千份提交代码，但其中许多程序可能仅在语法层面存在差异，而功能行为高度相似。通过将这些高度相似的程序进行聚类，并在每个族群中仅选择一个代表性程序执行，可以显著降低时间开销。因此，本文提出了一种基于动态特征的聚类算法（Dynamic Feature-based Clustering, DFC）。DFC 的核心思想在于提取程序中所有

变量的值序列，并以此作为动态特征的代表依据。若两个程序在现有测试数据集上的值序列完全一致，则 DFC 判定其动态特征相同。即便程序在语法结构上存在显著差异，它们仍可被归入同一簇，且在新增测试数据下预计保持一致的特征表现。

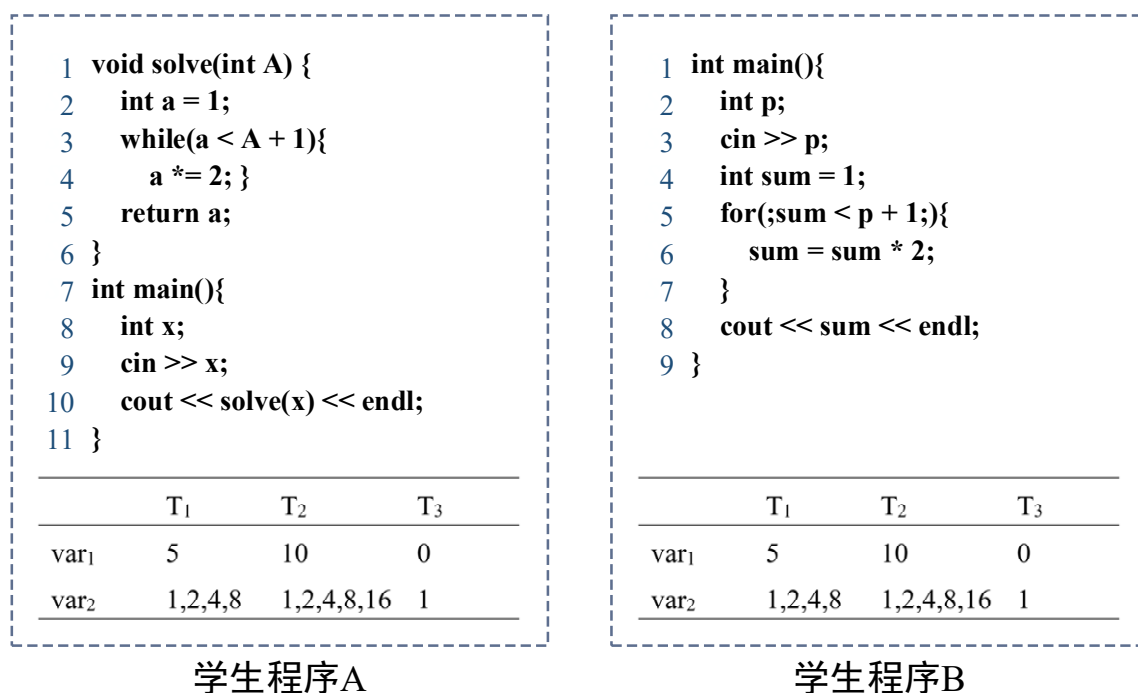


图 3-2 DFC 方法示例

Fig.3-2 An example of DFC approach

图 3-2 展示了 DFC 方法的一个示例。其中，学生程序 A 与学生程序 B 在语法结构上存在显著差异，二者采用了不同的代码组织方式。然而，这两个程序均仅包含两个变量。通过对变量名称进行模糊化处理后，可以发现，在全部测试数据下，二者的相应变量值序列完全一致。因此，学生程序 A 与学生程序 B 可以被聚类至同一族群。DFC 的工作流程如下所述：

（1）代码清理

使用 clang-format-15 命令对 C++源代码进行格式化。该过程确保代码风格一致，便于后续的插装操作。

（2）程序插装

DFC 利用 TreeSitter 解析原始程序的抽象语法树，定位涉及变量值变化的语句，并插入用于追踪的输出语句。具体来说，TreeSitter 从源代码生成抽象语法树，DFC 从根节点开始进行深度优先搜索，根据节点类型的不同采取相应的处理策略。

定义节点：对应变量的定义语句（如 `int a`），关键词为 `declaration` 或 `field_declaration`。对于此类节点，变量信息存储在其子节点中。因此，DFC 遍历其子节点，提取其节点类型及节点内容。随后，DFC 根据子节点的类型来解析节点内容，进而获得变量名称和变量的数据类型。其中，变量名称用于拼接输出语句及后续的追踪操作，数据类型

则用于避免在插装输出语句时，因插装了违反语法规则的语句而导致编译错误。子节点类型与其对应的属性和内容如表 3-2 所示：

表 3-2 节点信息
Table 3-2 Node information

子节点类型	子节点属性	内容
type_identifier	数据类型	基本数据类型，如 int、float、double
struct_specifier	数据类型	结构体
array_declarator	数据类型	数组
template_type	模板类型	模板
primitive_type	访问控制修饰符	如 private、public
placeholder_type_specifier	占位符类型	如 auto
identifier	变量名称	如 x、y、z
init_declarator	表达式	如 x = 1;
field_identifier	变量名称	如 x、y、z

通过遍历定义节点的子节点，DFC 获取了该定义语句所定义的变量名称与变量类型，并记录在字典中。其中，对于较为复杂的数据类型，DFC 进行了特殊处理。例如，DFC 通过深度递归来处理多维数组。以二维整形数组 x 为例，其变量名称为 x，数据类型被解析为 int[][]。对于结构体，DFC 将其中的变量与结构体名称进行了拼接。例如结构体 stu 中包含一个整形变量 x，则该变量的数据类型为 int，变量名称为 stu.x。

赋值节点：对应赋值语句（如 a = a + 1），关键词为 assignment_expression。对于赋值节点，DFC 读取该节点的左子节点来获取变量名称，随后对该进行检查。如果该变量为基本数据类型，则记录该节点下标，等待后续进行插装。

更新节点：对应自增或自减语句（如 a ++），关键词为 update_expression。对于此类节点，DFC 的处理策略与赋值节点一致。

初始化节点：对应初始化语句（如 a = 1），关键词为 init_declarator。对于初始化节点，DFC 的处理策略与赋值节点一致。

调用节点：对应函数调用（如 a.pop()），关键词为 call_expression。对于调用节点，DFC 首先会判断其属于普通函数调用还是成员函数调用。如果是普通函数调用，DFC 会选择忽略该节点。这是因为普通函数调用中的变量值变化通常发生在函数内部，无法直接在调用处体现。而如果是成员函数调用，情况则不同。成员函数调用往往涉及入栈/入队或出栈/出队等操作，这些操作会直接导致变量状态的改变，因此 DFC 会对其进行进一步处理。

在处理成员函数调用时，DFC 会通过识别子节点中的 function 关键词来提取与此次调用相关的变量名称。随后，DFC 将入栈和入队操作统一抽象为“+”，将出栈和出队操作抽象为“-”，而不去关注具体的数值。这是因为具体的数值并不会对聚类

结果产生影响,反而会增加存储成本。在绝大多数情况下,不同族群的程序在成员函数调用的行为和次数便存在显著差异。即使偶然情况下其行为和次数相同,DFC 依然能够依靠其他变量的值序列差异将这些程序有效区分开来。

在解析每个节点时,DFC 会记录涉及变量值变化的节点,并统计所有节点涉及变量的使用次数,用于后续步骤处理。在解析完整个语法树后,DFC 将输出语句插装到所有记录的节点之后。

(3) 变量值序列提取

在现有测试数据上运行插装后的程序,程序按执行路径输出变量值。DFC 通过关键字识别,将同一变量的值聚合成序列。由于 OJ 系统存在查重机制,部分用户可能插入无效代码以降低重复率,因此 DFC 根据上一步记录的使用次数,剔除使用次数为 0 的变量。随后,DFC 不再关注变量名称,而是将每个值序列视为字符串,并按字典序进行排序。最终,这些排序后的值序列共同构成该程序的动态特征。

(4) 聚类

第(3)步中得到的变量值序列会存储在字典中。对于每个新程序的变量值序列,DFC 查询字典中是否存在该序列。若存在,则将其归入对应族群;若不存在,则创建新族群,并将序列添加到字典中。通过这一过程,DFC 根据程序的动态特征实现程序聚类。

3.3.2 基于静态特征的差分测试

在 DFC 完成聚类后,本章节开始进行假阳性程序检测。其核心思想为从每个族群中随机挑选一个学生程序作为该族群的代表程序参与差分测试,并将该程序的测试结果赋予其所在族群的其他学生程序。进行差分测试时,现有研究在题目给定的输入约束范围内均匀生成了大量的测试数据,执行开销极大。然而,实验表明,在大多数题目中,约 70%的测试数据是冗余的,仅凭 30%的测试数据即可检测出接近 90%的假阳性程序。其原因在于大量随机生成的测试数据在程序执行逻辑上是等价的,未能有效提高测试覆盖率。进一步分析假阳性程序后发现,许多程序的错误集中在数组处理、数据类型定义及分支边界条件等方面。然而,现有测试数据集的完备性不足,难以有效发现此类错误。

为此,本章节提出了一种基于静态特征的差分测试方法(Static Feature-based Differential Testing, SFDT)。该方法的主要流程如下:

(1) 代码清理

使用 clang-format-15 命令对 C++源代码进行格式化。该过程确保代码风格一致,便于静态解析。

(2) 创建候选测试集

为了确保测试的完备性, SFDT 在题目输入范围的约束内生成覆盖性测试数据。根据 OJ 系统中的输入特点, 约束类型可分为三类:

- 数据类题目: 输入通常由大量的元素组成。例如题目 1070 的每行输入最多包含 20000 个整数。
- 数值类题目: 输入的元素数量较少, 但数值范围极大。例如题目 6498 中输入整数的最大值为 10^{18} 。
- 其他类型: 除上述两类外, 其他算法题目的输入规模通常是固定且有限的。

针对前两类题目, SFDT 通过遍历输入范围的指数上限来生成测试数据。例如, 对于题目 6498, 其输入范围为 1 至 10^{18} , SFDT 解析其指数上限为 18, 并在区间 $[10^n, 10^{n+1}]$ ($n=0, 1, \dots, 17$) 内随机生成 5 个测试数据。此外, 由于程序更容易在输入的上下边界范围内出现错误, 因此, SFDT 会在 $[10^0, 10^1]$ 与 $[10^{17}, 10^{18}]$ 内额外生成 3 个测试数据, 以便更有效地触发潜在错误。对于其他类型的算法题目, SFDT 采用均匀随机的方式生成了 30 个测试数据, 实验表明这是实现最佳效果的最小规模数据集。

(3) 补充候选集

SFDT 利用 TreeSitter 解析原程序的抽象语法树, 与 3.3.1 节所述方法不同, 其重点在于分析与输入变量相关的代码语句。具体来说, SFDT 首先遍历所有节点, 识别类型为 `expression_statement` 的表达式节点, 并检测其中是否包含 “scanf” 或 “cin” 关键词, 以定位输入语句。找到输入语句后, SFDT 通过解析关键词提取所有输入变量。

随后, SFDT 再次遍历 AST, 筛选出与输入变量相关的语句。对于涉及自然数判断的条件语句, SFDT 分析判断条件, 构造能够触发所有分支的测试输入, 并将其纳入候选测试集。对于输入语句, SFDT 检查变量的数据类型: 若为数组变量, 则生成超出数组上限的测试输入; 若为基本数据类型, 则构造边界值和溢出测试数据, 例如对 `int` 类型变量生成 `INT_MAX+1` 和 `INT_MIN-1` 等输入。这种方法通过精准定位和分析与输入相关的代码结构, 生成更具针对性的测试数据, 从而提升测试效率和覆盖率。

(4) 验证测试数据集

生成候选测试数据后, SFDT 检查其是否符合题目输入约束, 剔除不符合要求的用例, 以确保测试集的有效性。

(5) 差分测试

SFDT 遍历 DFC 聚类后的族群, 从每个族群中随机选取一个学生程序, 组成测试集。随后, SFDT 遍历测试数据集, 并在测试集中的所有程序上依次执行测试。对于每个测试数据, SFDT 对比所有程序的输出, 并采用多数投票机制判定结果: 输出占比最高的程序被视为正确程序, 与其输出不一致的程序则被标记为假阳性程序。

算法 1 展示了差分测试的实现过程, 其输入包括聚类后的族群 (GP) 和测试数据集 (TestCases)。在执行过程中, 算法首先随机从每个族群中选择一个学生程序 (第 1 行)。在迭代过程中, 算法获取每个程序的执行结果, 并统计结果出现的频次 (第

6-14 行)。执行全部程序后,算法采用多数投票机制,将出现次数最多的输出视为预期输出,并将输出不同结果的程序判定为假阳性程序(第 15-20 行)。之后,算法重复此过程,直到遍历完所有测试数据集(第 3-21 行)。最后,算法将每个程序的判定结果同步至其所在族群中的所有程序,并返回测试中被判定为假阳性的程序集合(第 22-23 行)。

算法 1 差分测试算法:

输入: GP: 聚类后的族群; TestCases: 测试数据集;

```

1. programs ← Random_Select(GP)
2. fpPrograms ← []
3. for testCase in TestCases do
4.     outputCount ← {}
5.     outputs ← {}
6.     for program in programs do
7.         output ← Execute_Program(program, testCase)
8.         outputs[program] ← output
9.         if output in outputCount then
10.            outputCount[output] += 1
11.        else
12.            outputCount[output] = 1
13.        end if
14.    end for
15.    majorityOutput ← Get_Max_Key(outputCount)
16.    for program in programs do
17.        if outputs[program] != majorityOutput then
18.            fpPrograms append program
19.        end if
20.    end for
21. end for
22. fpPrograms ← Reivew(GP, fpPrograms)
23. return fpPrograms

```

3.4 实验设计

3.4.1 研究问题

为了系统评估在线评测系统中现有测试数据的质量,并验证本章提出的假阳性程序检测方法在精度与效率方面的改进,本文构建了以下三个研究问题:

研究问题 1: 目前 OJ 系统中人工设计测试数据的有效性如何?

本问题旨在深入调研 OJ 系统中现有测试数据的充足性和完备性。为此,本文从多个维度对测试数据进行评估,包括代码覆盖率、变异得分和测试规模。代码覆盖率用于衡量测试数据对程序逻辑的覆盖程度,变异得分反映测试数据检测程序潜在错误的能力,而测试规模则体现其在实际应用中的辅助功能。通过这些指标的综合分析,揭示现有测试数据设计的优势与不足,为后续改进提供依据。

研究问题 2: 本文提出的基于聚类 and 差分测试的假阳性程序检测方法有效性如何?

本问题旨在全面评估本章节提出的聚类与差分测试相结合的假阳性程序检测方法的实际效果。对于聚类算法, 本实验通过比较聚类前程序总数与聚类后族群数量, 分析其在程序约简方面的能力。对于差分测试, 本实验通过统计检测出的假阳性程序数量, 评估方法的性能, 并进一步探讨现有测试数据的完备性。

研究问题 3: DFC-SFDT 与 DT 相比有什么提升?

本问题旨在系统比较本文提出的 DFC-SFDT 方法与传统 DT 方法在性能上的差异。实验主要聚焦检测精度和检测效率两个核心维度, 通过对比两种方法在假阳性程序检测中的表现, 评估它们在处理大规模程序集时的优劣。

3.4.2 实验对象

表 3-3 BuctOJ 题目信息

Table 3-3 Question information of BuctOJ

题目编号	程序数量	正确程序数量	错误程序数量	累计代码量	代码行数范围	测试数据数量
1069	2234	1106	1128	100178	18-151	6
1070	4415	1515	2900	348599	11-123	8
1119	2498	1480	1018	254077	14-265	2
2130	3219	1394	1825	145872	12-95	3
4254	2116	1580	536	101543	18-137	6
6498	1363	600	763	44925	16-172	15
总计	15845	7675	8170	995194		40

BuctOJ: BuctOJ 是北京化工大学在线评测系统, 被用于数据结构、算法设计等计算机相关课程的日常练习与考试之中。同时, BuctOJ 上会定期组织周赛、月赛等算法竞赛。在 10 余年的运行中, BuctOJ 积累了大量的学生程序与算法题目。这些学生程序常被用于错误定位、错误修复和测试数据生成等研究中。本文从 BuctOJ 的数据结构考试和算法竞赛中随机选取了 6 道算法题目, 其数据信息如表 3-3 所示。

表 3-4 AtCoder 题目信息

Table 3-4 Question information of AtCoder

题目编号	程序数量	正确程序数量	错误程序数量	累计代码量	代码行数范围	测试数据数量
abc364b	2862	1227	1635	124751	11-179	21
abc364c	2919	1130	1789	130238	10-181	30
abc365c	2930	923	2007	159608	21-196	28
abc366b	2992	1084	1908	132344	15-247	21
abc366c	3103	1102	2001	114148	23-208	25
abc373b	2554	1571	983	63709	9-155	28
总计	17360	7037	10323	724798		153

AtCoder: AtCoder 是来自日本的著名算法竞赛网站, 该网站会定期发布 AtCoder Regular Contest 和 AtCoder Beginner Contest 算法竞赛, 吸引了世界各地的竞赛选手共同参与。本文从 AtCoder 中随机挑选了 6 道题目, 并通过爬虫爬取了每道题目的正确程序与错误程序。题目的数据信息如表 3-4 所示。

3.4.3 评价指标

对于 OJ 系统现有测试数据的有效性调研, 本文选择了以下评价指标进行量化:

(1) 代码覆盖率^[10]:

代码覆盖率包含行覆盖率与分支覆盖率。行覆盖率衡量了语句级执行的完备性, 确保程序中的每一项基本操作均得到验证; 分支覆盖率则评估了控制流的全面性, 有效揭示条件判断中可能隐藏的缺陷。行覆盖率与分支覆盖率越高, 表明测试数据的检测范围更加广泛, 测试更加完备。

行覆盖率计算公式为:

$$Cov_{stmt} = \frac{L_{executed}}{L_{total}} * 100\% \quad (3-2)$$

其中, $L_{executed}$ 表示程序中执行过的代码行的数量, L_{total} 表示程序整体的代码行数。

分支覆盖率计算公式为:

$$Cov_{branch} = \frac{B_{executed}}{B_{total}} * 100\% \quad (3-3)$$

其中, $B_{executed}$ 表示程序中执行过的分支的数量, B_{total} 表示程序整体的分支数量。

(2) 变异得分^[11]:

变异得分为变异测试的执行结果, 反映了测试数据对程序变化的敏感程度。其计算公式如下:

$$Score = \frac{Mutant_{killed}}{Mutant_{total}} * 100\% \quad (3-4)$$

其中, $Mutant_{killed}$ 表示被测试数据成功“杀死”的变异体数量, $Mutant_{total}$ 表示生成的变异体总数。变异得分越高, 表明该测试数据集的错误检测能力越强, 能够更有效地捕捉到潜在的程序错误。

(3) 测试规模:

该指标依据第 3.2.3 节中提出的测试数据集、测试数据行数及测试元素数三个维度, 对测试数据的规模进行度量。较大的测试规模通常意味着学生在利用测试数据复现错误和定位问题时所需的成本越高。因此, 在具备良好错误检测能力的前提下, 测

试规模较小的测试数据集质量更高。

对于本章节提出的基于聚类 and 差分测试的假阳性程序检测方法，本文选择了以下指标进行量化：

(1) Precision^[70]：指被正确判定为假阳性程序的数量与被判定为假阳性程序的数量之比，计算公式为：

$$Precision = \frac{TP}{(TP + FP)} \quad (3-5)$$

Precision 值越高意味着检测的精度越高，误报的数量更少。

(2) Recall^[70]：指被正确判定为假阳性程序的数量与所有实际为假阳性程序的数量之比，计算公式为：

$$Recall = \frac{TP}{(TP + FN)} \quad (3-6)$$

Recall 值越高则漏报的数量越少。

(3) F-measure^[70]：为 Precision 值和 Recall 值的调和平均。F-measure 综合考虑了精确率与召回率，用于平衡两者之间的性能。其计算公式为：

$$F - measure = \frac{2 \times Precision \times Recall}{(Precision + Recall)} \quad (3-7)$$

(4) 时间开销：为各方法的执行时间，用于对比不同方法的执行效率。

3.4.4 基准方法

本文与 Liu^[61]等人使用的基于差分测试的假阳性程序检测方法进行了比较。具体来说，Liu 等人采用均匀随机算法（Uniform Random Algorithm, URA）来为每个题目生成 100 组测试数据。以 BuctOJ 平台上的 1070 题为例，该题要求处理一元多项式，每行输入代表一个多项式，其中数据项数量的上限为 10000。在这种情况下，URA 会执行以下步骤：

1、生成测试数据：在[1, 10000]范围内均匀随机生成 100 个整数，每个整数代表一个多项式的项数。

2、构建输入数据：基于生成的项数，为每个测试数据创建相应的多项式输入。

随后，DT 方法在这 100 组测试数据上执行差分测试。通过多数投票机制，该方法能够识别出在测试中表现异常的程序，即那些产生少数输出结果的程序。这些程序被视为假阳性程序。

3.5 结果分析

3.5.1 在线评测系统现有测试数据质量分析

针对研究问题 1，本章节从 3.2 节论述的框架出发，全面衡量了 OJ 系统人工设计的测试数据的质量。

(1) 代码覆盖

表 3-5 OJ 系统人工设计测试数据的代码覆盖率

Table 3-5 Code coverage of manually designed test data in OJ system

题目编号	行覆盖率	分支覆盖率	正确程序行覆盖率	正确程序分支覆盖率
1069	76.43%	84.42%	79.02%	86.02%
1070	81.11%	85.46%	81.72%	84.61%
1119	80.91%	73.14%	84.89%	71.44%
2130	83.05%	78.77%	83.82%	77.48%
4254	84.27%	75.28%	85.29%	75.14%
6498	80.10%	72.37%	82.35%	75.86%
abc364b	85.28%	82.78%	88.38%	79.50%
abc364c	83.59%	77.15%	87.12%	72.38%
abc365c	78.71%	90.39%	77.25%	87.58%
abc366b	81.22%	80.59%	84.08%	75.89%
abc366c	80.30%	79.73%	83.05%	78.57%
abc373b	85.53%	72.16%	86.00%	68.54%

本实验首先统计了现有测试数据在学生程序上的行覆盖率与分支覆盖率，结果如表 3-5 所示。其中，第一列为题目编号，第二、三列分别展示了题目的行覆盖率与分支覆盖率。此外，本实验还单独统计了正确程序代码覆盖情况，结果展示在第四、五列。

从表 3-5 可以看出，OJ 系统中人工设计的测试数据在代码覆盖率方面仍存在一定局限性。具体来说，测试数据的行覆盖率分布在 76.43%至 85.53%之间，平均行覆盖率为 81.71%；分支覆盖率则在 72.16%至 90.39%之间波动，平均分支覆盖率为 79.35%。值得注意的是，即使是行覆盖率最高的题目（abc373b），其覆盖率也仅达到 85.53%，未能突破 90%。这表明当前测试数据集在覆盖代码路径方面仍然存在较大改进空间，部分代码行和分支未被充分测试。

由于测试数据只能检测其覆盖路径上的错误，覆盖率偏低可能会导致部分错误未被检测到，从而影响测试数据的有效性。从表 3-5 的第四、五列数据可以观察到，正确程序的行覆盖率与分支覆盖率均低于 90%，这表明现有测试数据可能存在漏判的风险，即部分错误程序可能未被正确识别。

为深入研究覆盖率不足的原因，本文对 200 份覆盖率低于平均水平的学生程序进

行了手动分析。除因测试数据完备性不足，导致未能覆盖的代码以外，本文还总结出四类主要的未覆盖代码：（1）无效分支：学生在编写程序时，因对算法题目的理解有误，创建了在当前输入约束中不可能触发的分支条件，这导致了程序的行覆盖率与分支覆盖率下降。（2）反查重代码：为了避免学生抄袭代码直接完成编程作业，OJ 系统通常都具有查重机制。部分学生为了对抗查重机制，往往会在程序中插入大量无关代码，这些代码难以被测试数据覆盖，导致覆盖率降低。（3）模板代码：在编程作业或竞赛训练中，为了提高编码效率、简化代码，学生往往会保存一套模板代码，并应用在算法题目中。例如通用的 I/O 解析器、常用的数据结构和算法等等。然而，大多数模板代码并不会发挥作用，反而会降低测试数据的覆盖程度。（4）调试代码：为了方便调试程序，学生通常会在程序中加入调试代码，例如文件输入与输出。然而，学生可能在调试完成后忘记删除此类代码，或者因为调试代码不影响程序的输出结果而保留，导致了代码覆盖率的下降。

（2）变异得分

本实验统计了人工测试数据在所有程序上的变异得分，结果如表 3-6 所示。其中，第一列为题目编号，第二列为原程序的数量，第三列为生成变异体的数量，第四列为杀死变异体的数量，第五列为变异得分。

从表 3-6 可以看出，现有测试数据在大多数题目上的变异得分超过了 0.8，部分题目甚至达到 0.9 以上。这表明现有测试数据集具备良好的错误检测能力，能够有效识别大部分潜在错误。特别是题目 6498，其 0.92 的变异得分反映了该题目的测试数据在错误检测方面表现最为出色。

表 3-6 OJ 系统人工设计测试数据的变异得分

Table 3-6 Mutation score of manually designed test data in OJ system

题目编号	原程序数量	变异体数量	被杀死的变异体数量	变异得分
1069	2234	337232	297062	0.88
1070	4415	1540980	1250270	0.81
1119	2498	395200	232451	0.59
2130	3219	651094	591682	0.91
4254	2116	279759	167550	0.6
6498	1363	150196	138778	0.92
abc364b	2862	726251	630750	0.87
abc364c	2919	417481	355138	0.85
abc365c	2930	422250	296050	0.7
abc366b	2992	518512	426357	0.82
abc366c	3103	253556	211059	0.83
abc373b	2554	215219	193092	0.9

然而，尽管现有测试数据集的整体表现良好，但是这些数据也暴露了人工设计的

局限性。即使是变异得分最高的题目 6498，仍然有 7.6%（11418 个）的变异体未能被杀死，这与理想状态下能够检测所有变异体的目标仍有一定差距。此外，不同题目间的变异得分差异显著，例如题目 4254 的得分仅为 0.6，远低于其他题目。表明该题目的测试数据质量相对较低。这种差异性突显了人工设计测试数据的不稳定性，难以作为所有题目生成高质量的测试数据。

（3）测试规模

对于测试规模，本章节从 3.2.3 节描述测试数据集、测试数据行数和测试元素数三个方面进行了统计，结果如表 3-7 所示。

可以看出不同题目的测试数据集规模存在显著差异。例如，abc364c 拥有 30 组测试数据，覆盖更广，而 1119 仅有 2 组，但单个测试数据规模较大大。平均每道题目包含 16 组测试数据。庞大而繁多的测试数据在增强错误检测能力的同时，也提高了学生发现错误后调试的成本。在测试数据行数方面，题目 abc366c 远高于其他题目，达到了 6316707 行，并且其平均每个测试数据的行数达到了 252668。对于初学者来说，失败的判定反馈与繁杂的测试数据会严重打击其学习的积极性。在测试元素方面，大多数题目都包含海量的测试元素。有 6 道算法题目的平均测试元素数量都超过了 1000。此类测试数据已经难以应用到调试当中，会给学生 debug 带来极大的困难。6498、abc372b、abc373b 等题目虽然在平均测试数据行数和平均测试元素数这两个指标上表现的十分友善，但分析题目描述后可知，这些题目的输入包含较少的元素，而元素的数值极高，调试的难度与前者相当。例如，题目 6498 的输入包含三个整数，每个整数的取值范围为 1 至 10^{18} 。其中一组测试输入为 [564659917154925708 372619853354233884 843027069177544681]。

表 3-7 OJ 系统人工设计测试数据的测试规模

Table 3-7 Test scale of manually designed test data in OJ system

题目编号	测试数据集	测试数据行数	测试元素数	平均测试数据行数	平均测试元素数
1069	6	12	8032	2	1338
1070	8	24	12230	3	1528
1119	2	207	411	103	205
2130	3	208	105668	69	35222
4254	6	12	257	2	42
6498	15	30	60	2	4
abc364b	21	878	941	41	44
abc364c	30	120	10587934	4	352931
abc365c	28	84	4504172	3	160863
abc366b	21	2651	2651	126	126
abc366c	25	6316707	9200057	252668	368002
abc373b	28	56	56	2	2

在 OJ 系统中,为了全面检测学生程序在不同边界条件下的正确性,通常会设计多样化的测试数据。本章节基于测试数据的输入规模和数据范围,将其划分为功能型测试数据和性能型测试数据,并进行了更细致的分析。

功能型测试数据:此类测试数据通常规模较小,输入元素较少或数据取值范围较低,主要用于验证学生程序是否满足基本功能需求。对于数据类题目,当测试数据的元素数量小于或等于 100 时,本文将其归类为功能型测试数据。对于数值类题目,则以整数 $N \leq 10000$ 或字符串长度小于或等于 50 作为划分标准。

性能型测试数据:该类测试数据通常规模较大,输入元素数量极多,或者数据取值范围巨大,旨在测试程序在极端情况下的正确性及其运行效率。对于数据类题目,若某个测试数据的元素总数超过 100,则归类为性能型测试数据。对于数值类题目,若某个输入的数值超过 10000,则该测试数据被归类为性能型测试数据。

本章节对两类测试数据分别进行了统计。结果如表 3-8、3-9 所示。

可从表 3-8 可以看出,全部题目都包含功能型测试数据,且在 BuctOJ 上,功能型测试数据的占比较高,平均占比为 52.5%。这表明管理员在测试数据设计时,更加关注程序的基本功能验证,同时兼顾初学者的编程体验。而在 AtCoder 上该比值远低于 BuctOJ,这是因为作为专业的算法竞赛平台,验证参赛选手的程序在各种边界情况下的正确性及执行效率更为重要。然而,题目 abc364b 和题目 abc373b 的功能型测试数据达到 100%。这是因为该题目涉及的算法较为复杂,因此输入被限定在很小的范围之内。例如题目 abc373b 的输入仅为一个包含 26 个小写字母的字符串。

表 3-8 OJ 系统中的功能型测试数据

Table 3-8 Functionality-oriented test data in OJ system

题目编号	总测试数据集	功能型测试数据集	测试数据行数	测试元素数	平均测试行数	平均测试元素数	测试占比
1069	6	4	8	30	2	7	66.67%
1070	8	6	18	270	3	45	75.00%
1119	2	1	5	9	5	9	50.00%
2130	3	1	4	15	4	15	33.33%
4254	6	5	10	155	2	31	83.33%
6498	15	1	2	4	2	4	6.67%
abc364b	21	21	878	941	41	44	100.00%
abc364c	30	3	12	46	4	15	10.00%
abc365c	28	4	12	30	3	7	14.29%
abc366b	21	5	309	309	61	61	23.81%
abc366c	25	2	23	34	11	17	8.00%
abc373b	28	28	56	56	2	2	100.00%

从表 3-9 可以看出,除题目 abc373b 与题目 abc364b 这类特殊题目以外,所有算法题目都设计了性能型测试数据,并且此类测试数据的平均测试元素数或元素通常远

超功能型测试数据。其中,绝大多数的性能型测试数据平均测试元素数都超过了 1000。虽然部分题目的测试元素数较少,其测试数据的数值极高。此类测试数据几乎无法为学生提供判题外的帮助。此外,即使部分算法题目的性能型测试数据占比较低,但是经手动分析表明,大多数的错误程序是由性能型测试数据检判定。因此,学生还是必须面对这些规模庞大、数值过高的测试数据。

表 3-9 OJ 系统中的功能型测试数据

Table 3-9 Performance-oriented test data in OJ system

题目编号	总测试数据集	性能型测试数据集	测试数据行数	测试元素数	平均测试行数	平均测试元素数	测试占比
1069	6	2	4	8002	2	4001	33.33%
1070	8	2	6	11960	3	5980	25.00%
1119	2	1	202	402	202	402	50.00%
2130	3	2	204	105653	102	52826	66.67%
4254	6	1	2	102	2	102	16.67%
6498	15	14	28	56	2	4	93.33%
abc364b	21	0	0	0	0	0	0.00%
abc364c	30	27	108	10587888	4	392144	90.00%
abc365c	28	24	72	4504142	3	187672	85.71%
abc366b	21	16	2342	2342	146	146	76.19%
abc366c	25	23	6316684	9200023	274638	400001	92.00%
abc373b	28	0	0	0	0	0	0.00%

3.5.2 本文方法的有效性分析

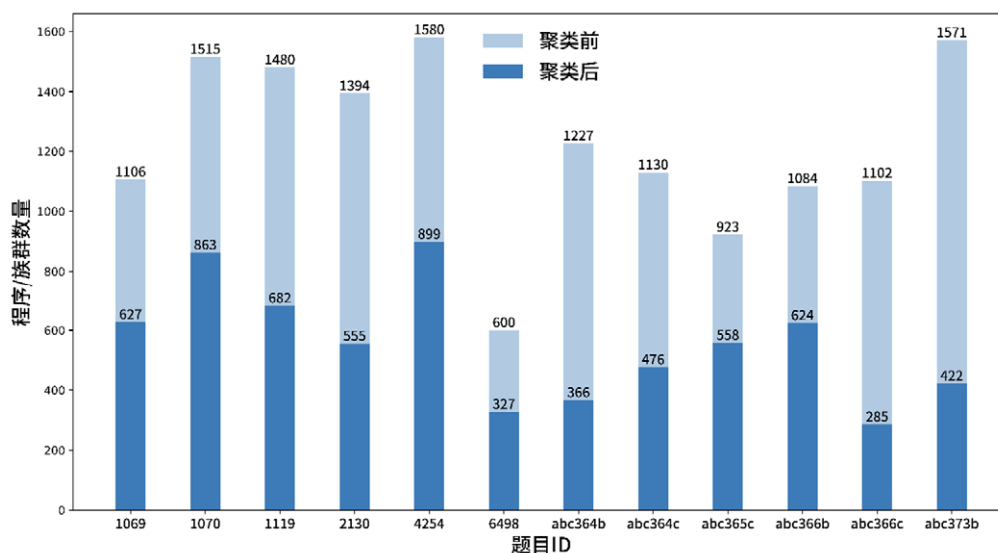


图 3-3 DFC 聚类结果

Fig.3-3 DFC clustering results

方法 DFC 的聚类结果如图 3-3 所示。图中纵轴表示程序或族群数量,横轴表示题

目 ID。每个柱形图表示该题目的聚类结果，浅色部分为聚类前的程序数量，深色部分为聚类后的族群数量。可以看出在所有题目上，大量的学生程序被聚类到了少量的族群中。14712 份学生程序被聚类为了 6684 个族群，平均有 54.57% 的程序被约减掉。在题目 abc366c 上，聚类的效果最佳，达到了 74.14%。这是因为该题目难度较低，学生程序代码短，且解法较为固定，动态特征更容易保持一致。而在更复杂的题目 abc365c 上，本章节的方法仍能达到 39.54% 的聚类效果。

随后，本章节基于聚类后的结果进行 SFDT，实验结果如表 3-10 所示。可以看出所有的题目都存在假阳性程序。在 14712 份正确程序中一共存在 2350 份假阳性程序。因此，漏判是一种普遍存在的现象。在 BuctOJ 系统中，平均每道题目有 204 份假阳性程序，比例为 20.35%。最严重的为题目 1070，接近半数的程序都存在未被检测出的错误。严重的漏判可能导致学生错误认知，为以后的研究和工作的埋下隐患。即使是更为专业的 AtCoder 系统，正确程序中仍存在 15.98% 的假阳性程序。因此，检测并解决漏报现象是 OJ 系统研究中的一个关键问题。

表 3-10 SFDT 假阳性程序检测结果

Table 3-10 SFDT false-positive program detection results

题目编号	程序数量	假阳性程序数量	假阳性率
1069	1106	116	10.49%
1070	1515	765	50.50%
1119	1480	22	1.49%
2130	1394	75	5.38%
4254	1580	63	3.99%
6498	600	66	11.00%
abc364b	1227	40	3.26%
abc364c	1130	35	3.10%
abc365c	923	130	14.08%
abc366b	1084	157	14.48%
abc366c	1102	22	2.00%
abc373b	1571	331	21.07%

3.5.3 本文方法与基准方法的效果对比

本文将 DFC-SFDT 与 DT 进行了对比。为了减少实验误差，本章节对两种方法均进行了五次重复实验，并将五次实验的平均结果作为最终的实验结果。

图 3-4 展示了两种方法的时间开销，图中横轴为题目编号，纵轴为时间开销（单位：秒）。红色实线与蓝色实线分别表示 DFC-SFDT 和 DT 的方法耗时。此外，本实验还单独统计了 DFC 和 SFDT 的时间开销，分别用橙色虚线与绿色虚线表示。可以看出，在绝大多数部题目上，DFC-SFDT 的时间开销都要少于 DT。具体而言，DT 在

12 道题目上的总时间开销为 72722 秒，而 DFC-SFDT 仅用时 36079 秒，整体减少了 50.39% 的耗时。其中，在题目 abc372b 上，DFC-SFDT 表现出最佳的优化效果，节省了 78.78% 的时间开销。而在题目 abc364b 和题目 abc366b 上，DFC-SFDT 的优化效果相对有限。在题目 4254 上，DFC-SFDT 甚至比 DFC 增加了 145 秒的时间开销。

为探究这一现象，本文对相关题目进行了手动分析。结果表明，这些题目的输入范围较小。其中输入区间最大的题目 4254，其输入上限仅为 100。对于此类题目，DT 的执行效率较高，而 DFC-SFDT 在程序分析阶段耗费了较多时间，导致整体效率未达预期。然而，此类小规模输入的题目在 OJ 系统中占比较低。并且随着 OJ 系统的运行，算法题目的提交程序不断积累，DFC-SFDT 在聚类上的优势将会更加明显。

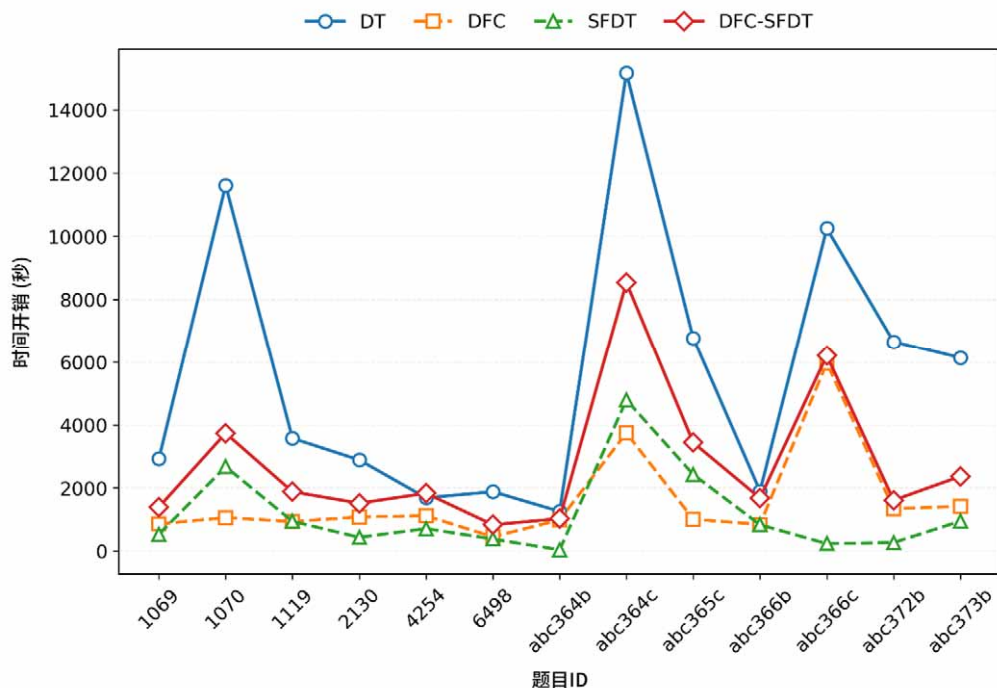


图 3-4 DFC-SFDT 与 DT 的时间开销对比

Fig.3-4 Time overhead comparison of DFC-SFDT and DT

在假阳性程序检测结果方面，本章节通过维恩图可视化展示了两种方法的结果分布。如图 3-5 所示，该图由两个相交圆形构成，其中左圆淡蓝色区域代表 DT 方法单独检测到的假阳性程序数量，右圆浅绿色区域表示 DFC-DFDT 方法特有的检测结果，中央深色交集区域则表示两种方法共同检测出的假阳性程序数量。可以看出 DFC-DFDT 总共检测出 2503 个假阳性程序，DT 则检测出了 2451 个假阳性程序。其中，约有 95.88% 的假阳性程序被两种方法全部检测到了。

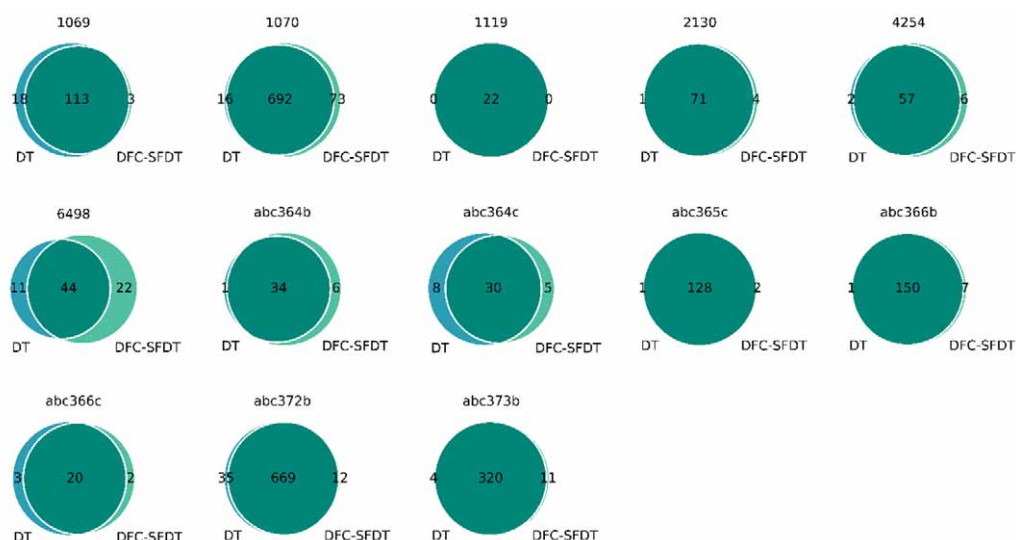


图 3-5 DT 与 DFC-SFDT 假阳性程序检测结果

Fig.3-5 DT and DFC-SFDT false-positive program detection results

表 3-11 展示了两种方法在 Precision、Recall 和 F-measure 三种指标上的表现。可以看出 DT 方法的 Precision 较高，在所有程序上都不存在误报的情况。而 DFC-SFDT 在 Recall 和 F-measure 均高于 DT 方法 0.01。因此，总体上看，本文提出的 DFC-SFDT 仅使用 DT 方法一半的时间开销，取得了与 DT 方法几乎一致的实验效果。

表 3-11 DT 与 DFC-SFDT 的指标分析

Table 3-11 Metric analysis of DT and DFC-SFDT

方法	Precision	Recall	F-measure
DT	1	0.95	0.97
DFC-SFDT	0.99	0.96	0.98

3.6 本章小结

本章节首先分析了 OJ 系统中的题目特征，指出人工设计测试数据的局限性，并从代码覆盖率、变异得分和测试规模三个角度出发，调研了 OJ 系统中现有的测试数据质量。随后针对 OJ 系统中的漏报问题提出了一种基于聚类和差分测试的假阳性程序检测方法，以实现更高效的假阳性程序检测。在研究问题 1 中，本文在 BuctOJ 和 AtCoder 收集的学生程序数据集上进行调研，验证了现有测试数据存在缺陷。在研究问题 2 中，DFC-SFDT 将 16290 份学生程序聚类为 7582 个族群，约简了 53.46% 的学生程序，并检测出 2350 个假阳性程序。研究问题 3 将 DFC-SFDT 与 DT 方法进行对比，DFC-SFDT 在仅使用 DT 方法 49.61% 的时间开销，而实现了更高的 Recall 和 F-measure 值，进一步证明了 DFC-SFDT 方法的有效性。

第四章 基于大语言模型和多目标优化算法的测试数据生成技术

本章对前文关于 OJ 系统中人工测试数据的评估结果进行了分析，提出一种基于大语言模型和多目标优化算法的测试数据生成技术。该方法旨在自动生成具有高错误检测能力的测试数据，同时兼顾测试规模。

4.1 研究动机

在 OJ 系统中，高质量的测试数据对于准确评估学生程序至关重要。然而，本文第三章揭示了当前人工设计测试数据存在的系统性缺陷：测试数据覆盖率不足、变异得分偏低以及测试规模失控等，暴露出传统方法的瓶颈。这些问题在数据结构与算法类题目中尤为突出，因其复杂的路径逻辑与严苛的输入约束对测试数据提出了更高要求。测试数据不仅需要确保程序执行路径的全覆盖，还要精准控制输入参数的合法边界，同时捕捉隐蔽的异常场景。这种多维度的质量要求严重降低了人工设计的效率，管理员往往需要投入大量时间，却难以平衡测试集的完备性与规模控制。这种低效模式不仅推高了 OJ 系统的运维成本，更制约了题库的迭代更新速度。

针对上述挑战，学术界已提出多种解决方案。基于 LLM 的测试数据生成方法虽能理解题目语义并产生多样化测试输入，但受限于输出 token 数量，无法生成极限情况下包含上千万数字或字符的测试数据。并且在生成过程中，LLM 常因忽视输入约束导致测试数据生成失败。

基于遗传算法的测试数据生成技术虽然可以针对性提升特定指标，部分研究聚焦于单一维度优化^[71]，或追求极致的错误检测率，或是代码覆盖率的最大化，难以实现多维属性的动态平衡。并且随机生成的初始种群质量难以保证，可能导致遗传算法陷入局部最优解^[72]。

然而，如果将两者相结合，利用 LLM 优秀的语义理解能力以及代码生成能力为遗传算法生成高质量的初始种群，利用遗传算法的迭代优化能力不断提升测试数据的质量。这种方法不仅能够充分发挥两种技术的优势，还能有效规避它们各自的局限性。因此，本章节提出了一种新的测试数据生成方法，结合了 LLM 和多目标优化算法的优势，生成兼顾错误检测能力和测试规模的高质量测试数据。

4.2 基于大语言模型和多目标优化算法的测试数据生成方法

图 4-1 展示了本文提出的基于大语言模型和多目标优化算法的测试数据生成方法（Large Language Model & Multi-Objective Optimization Algorithm-Based Test Data

Generation Approach, LLM-MOGA)。该方法构建了"LLM 预生成-遗传算法优化"的双阶段架构：第一阶段通过 LLM 解析题目语义生成测试数据生成器，第二阶段采用多目标优化算法对生成的测试数据进行质量提升。相较于传统遗传算法依赖人工编写算法构建初始种群的方式，本方法通过 LLM 自动生成初始种群，在保证种群多样性的同时显著降低了人工参与成本。

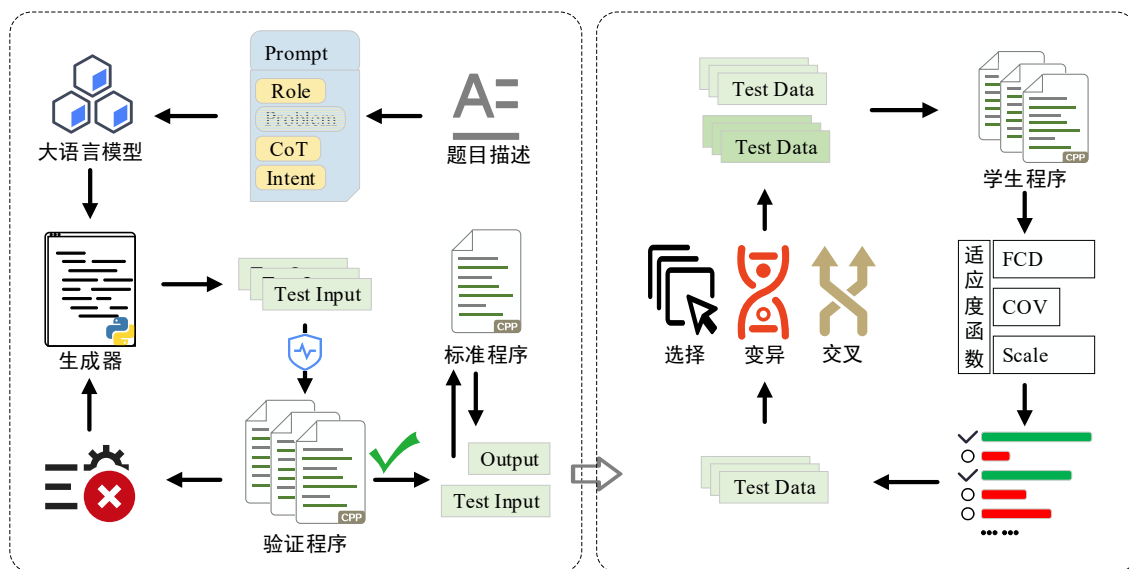


图 4-1 LLM-MOGA 方法流程图

Fig.4-1 Flowchart of the LLM-MOGA approach

4.2.1 大语言模型预生成测试数据

LLM 预生成测试数据的核心流程分为三个阶段：输入生成；验证及修复；输出生成。由 4.1 的分析可知，LLM 在直接生成测试数据方面的表现不佳。因此，基于其出色的语义理解能力和代码生成能力，本文使用 LLM 阅读题目描述并生成 python 形式的测试数据生成器，通过执行该生成器进而生成测试数据，减轻了其语义负担。

在输入生成阶段，本文首先为 LLM 构建了虚拟角色，现有研究表明这可以显著增强 LLM 在特定任务上的理解能力^[73]。随后，本文将题目的自然语言描述转换为结构化的提示模板，包括输入格式、边界约束等信息，并将该模板信息与示例输入给 LLM，令其充分理解题目需求。由于 LLM 缺乏连贯的逻辑推理能力，因此本文使用思维链来引导 LLM 进行多步逻辑思考：

(1) 约束解析：要求 LLM 分析、提取题目中的输入约束，如输入格式（数据类型、变量个数）、边界条件等，并以自然语言的形式进行描述。

(2) 规则转换。要求 LLM 将约束信息转换为代码逻辑，例如将“整数取值范围 $1 \leq n \leq 10^5$ ”映射为生成器中的 `random.randint(1, 100000)`调用；

(3) 生成构造函数。要求 LLM 生成 python 函数形式的参数化测试数据生成器，

支持通过参数控制测试数据的测试规模。

提示词设计如下所示。

LLM 预生成提示词设计：

角色设计：你是一个专业的算法竞赛选手以及编程教师，擅长编写算法题目和测试数据。

思维链：接下来我会给你提供一个算法题目以及输入要求，你需要完成以下任务：

1、分析题目中的输入约束，例如输入的类型、个数、范围等

2、将约束信息转换为代码逻辑

3、整理所有代码逻辑，编写 python 函数，要求该函数包含一个参数 `scale`，该参数可以控制测试数据的规模，执行该函数可以生成符合题目约束的输入数据

你只需要给我提供最终生成的 python 函数即可

问题描述：

输入格式：

输入示例：

在生成测试数据生成器后，本文采用了一种均匀生成策略来生成测试输入。具体来说，对于包含 X 个输入范围为 1 至 N 的算法题目，本文首先将 `scale` 的值设为 1 和 N 传入生成器中，分别生成覆盖两个边界的测试输入。随后再从 1 至 N 中均匀生成 $X-2$ 个整数值作为 `scale` 传入生成器中。该策略最终得到的测试数据集相比随机生成的覆盖范围更加广泛。

在验证及修复阶段，验证算法从数据集中随机挑选 10 个正确程序为测试集，并执行 LLM 输出的生成器来随机生成 10 组测试输入。随后验证算法在测试集中执行所有的测试数据，并基于错误信息及输出一致性来检验生成器。具体来说，若测试集中部分程序在执行测试数据时出现运行错误、数组越界等错误信息时，验证算法会判定该生成器存在错误导致其生成的测试输入非法。即使测试输入在所有程序中都成功执行，验证算法仍然会对比测试集的输出结果。这是因为当测试输入非法时，测试集的程序可能由于兼容性较强而执行成功，但非法输入可能触发错误的分支而导致最终输出预期之外的结果。因此，当测试集中的程序出现不同输出时，验证算法会判定生成器错误。如果生成器最终被判定为正确，则进入输出生成阶段。否则验证算法会将该生成器反馈给 LLM 并提示该生成器包含错误，令 LLM 尝试修复该生成器。

在输出生成阶段，由于 OJ 系统中的算法题目通常都存在标准程序，本文在该程序上执行测试输入 Ini ，收集测试输出 $Output_i$ ，并组合为完整的测试数据 $(Ini, Output_i)$ 。这种输入输出生成解耦的设计降低了 LLM 的语义理解负担，避免了传统方法中 LLM 需同时生成输入输出的复杂性，同时大幅提高了测试数据生成的成功率。

4.2.2 多目标优化算法持续优化

(1) 预处理

在遗传算法实施前,本文建立了系统化的预处理框架。OJ 系统题目描述中通常包含输入参数的完整约束规范,预处理阶段提取了输入的三个数据:输入参数类型(数值型/字符型/复合型)、规模约束以及值域约束。这些值用于在遗传算法对个体的编码及衡量测试规模。为保证与人工设计测试数据的对比公平性,本文记录了现有测试数据集的基数作为基准参数并采用等量对照原则:将遗传算法种群规模设置为与人工测试数据数量严格一致。以题目 1070 为例,其输入要求处理两个一元多项式,每个多项式以由系数和指数组成的元组序列降幂排列,终止符为“-1 -1”。其中指数值域为正整数且数据总量不超过 10000 项。在预处理框架中,本文提取了“整数”的参数类型、“指数大于 0”的值域约束和“数据量小于 10000”的规模约束。

(2) 多维评估

在多目标优化算法中,适应度函数用来评价个体的优劣。适应度函数的设计决定了种群的进化方向,是解决实际问题的关键因素。在 OJ 场景中,错误检测是测试数据的直接用途。OJ 系统的首要目标便是为用户提供一个准确、高效的反馈平台,从而减轻教师的工作压力。因此,错误检测能力被作为 LLM-MOGA 适应度函数的核心目标。

其次,OJ 系统作为广泛应用于编程教学的在线评测平台,其错误反馈不仅要高效,还要具有一定的教育价值。学生在收到程序错误反馈后,往往会下载失败的测试数据,通过复现错误来分析程序中的问题。虽然在部分情况下,规模较大的测试数据可以帮助学生深入思考程序在边界情况下的处理方式,但测试数据规模过大却可能适得其反。尤其是在初学者阶段,庞大的测试数据不仅无法为学生提供有效帮助,反而可能导致学生的学习兴趣下降。因此,测试规模被视为 LLM-MOGA 适应度函数中的次要目标,旨在保证生成的测试数据既具备足够的覆盖性,又不至于冗长。

接下来,本文将详细介绍两个适应度函数的计算方法。

错误检测能力: 本文使用了综合代码覆盖率与错误检测率的测试数据评估方法,旨在更全面地衡量测试数据的错误检测能力。尽管变异得分在测试数据评估中具有良好的有效性,但由于其计算过程中变异测试的时间开销过大,不适用于遗传算法的迭代过程。

在软件测试领域,基于行覆盖率与分支覆盖率的代码覆盖率作为评估测试数据质量的基础指标被广泛采用,理论上行覆盖率、分支覆盖率越高的测试数据,其错误检测能力更强。这种评估方式虽然简单直观,却也存在明显的局限性。首先,该指标仅关注“是否覆盖”的二元信息,忽略了覆盖频次等动态执行信息。其次,其未考虑覆盖路径的多样性并对程序中的冗余代码敏感,导致评估结果容易出现偏差。

程序示例:

```
1.  #include <iostream>
2.  using namespace std;
3.  int main(){
4.      int a, b;
5.      cin >> a >> b;
6.      if(a < 5){
7.          cout << a << endl;
8.      }
9.      else{
10.         if(b < 5){
11.             cout << b << endl;
12.         }
13.         else{
14.             int x = a + b;
15.             ... .. (20 行调试代码)
35.             cout << x << endl;
36.         }
37.     }
38. }
```

以上述程序为例,其中 15 至 34 行为调试代码,对解决算法问题并无实质性贡献,属于无效代码。此时,测试数据集合 A ([3, 4]、[5, 4]) 的行覆盖率为 36.84%,分支覆盖率为 66.67%。测试数据集合 B ([5, 4]、[5, 5]) 的行覆盖率为 94.74%,分支覆盖率为 66.67%。可以看出由于调试代码的存在,在传统的代码覆盖率指标上测试数据集 B 显著优于集合 A。然而,如果剔除掉无效代码后重新评估,集合 A 与集合 B 的有效代码覆盖方面具有同等的价值。这一现象在 OJ 系统中尤为普遍,3.4.1 节的实验结果表明 OJ 系统中学术程序的行覆盖率与分支覆盖率普遍偏低,其主要原因之一在于学生程序通常包含大量的调试代码、模板代码和为规避查重而添加的冗余代码。因此,降低此类代码对评估结果的影响,同时增强覆盖的多样性成为提高该指标完备性的关键问题。为此,本文提出了一种基于覆盖频次的动态权重调整策略(Coverage-Frequency Based Dynamic Weighting Strategy, CFDW),其核心思想是通过动态调整不同程序路径的权重,鼓励测试数据覆盖更多样化的程序执行路径,尤其是较少被执行的路径。

具体来说,在遗传算法的迭代过程中,CFDW 记录了测试数据在程序上的覆盖路径,并结合程序 ID 进行唯一编码,以确保路径信息的精确追踪。对于每条执行路径,系统记录其累积覆盖频次,作为后续权重调整的依据。随后,CFDW 采用分阶段的权重调整策略:

初始阶段(前 50 次迭代):所有覆盖路径的权重统一设为 1.0,系统仅记录覆盖路径及其频次,不进行权重调整。遗传算法初期生成的测试数据质量普遍较低,此时频繁被覆盖的路径往往是无效代码或程序主流程代码,对提升测试质量的贡献有限。

权重调整阶段（迭代 50 次后）：CFDW 基于累积的覆盖频次信息开始动态调整路径权重。对于路径 $Path_i$ ，其权重计算为覆盖频次取对数后的倒数：

$$W(Path_i) = \frac{1}{\log_2 Fre_i} \quad (4-1)$$

随后，对所有被覆盖路径的权重进行归一化，并乘以被覆盖路径总数：

$$Norm_W(Path_i) = \frac{W(Path_i)}{\sum_{j=1}^{Num_{cov}} W(Path_j)} * Num_{cov} \quad (4-2)$$

其中， Fre_i 表示路径 $Path_i$ 的覆盖频次， Num_{cov} 表示被覆盖的路径总数。

CFDW 设计了两种权重更新触发机制：定期更新和新路径覆盖更新。每当遗传算法迭代 50 次时，CFDW 会重新计算所有覆盖路径的权重，以反应累积覆盖情况的变化。此外，当测试数据覆盖到新的程序路径时，会立即重新计算所有路径权重。为了避免某条覆盖路径长期占据过高的权重，或路径被初次覆盖时，其对数值为 0 的问题。在每次迭代时，CFDW 会为所有路径的覆盖次数加 1。这样既确保了新发现路径不会获得过高权重，也保证了其对数值不为 0。基于上述权重调整策略，个体在代码覆盖目标上的适应度计算公式定义为：

$$C_p = \frac{\sum_{i=1}^{Num_{bran}(p)} Norm_W_i \cdot \delta_i}{Num_{bran}(p)}, \quad Fit_{cov} = \frac{1}{Num_{pro}} \sum_{p=1}^{Num_{pro}} C_p \quad (4-3)$$

其中 Num_{bran} 表示该程序的全部可能执行路径， δ_i 表示该路径是否被覆盖，若被覆盖则为 1，未被覆盖则为 0。这种基于覆盖频次的动态权重评估方法与传统的代码覆盖率指标相比，不仅能够有效降低无效代码对评估结果的干扰，还能通过鼓励覆盖多样性来提高测试数据发现潜在错误的能力，从而为测试数据生成提供更精确的指导。

除代码覆盖率外，错误检测率是评估测试数据错误检测能力的另一核心指标。基于第三章实验成果，本文在错误检测率计算中引入了对假阳性程序的特殊处理机制。在 OJ 系统中，由于漏报现象的存在，部分被判定为“正确”的程序实际上包含隐藏错误。这些假阳性程序即使在人工设计的测试数据集下也未能被检出，具有更高的检测价值。因此，本文将识别出的假阳性程序纳入错误程序集，并赋予其 2.0 的权重（其他错误程序权重为 1.0），错误检测率计算公式如下：

$$Fit_{Fcd} = \frac{\sum_{i=1}^N Check_i * W_i}{Num_{fail}} \quad (4-4)$$

其中, $Check_i$ 表示错误程序 i 是否被检测出 (检测出为 1, 未检测到为 0), W_i 表示错误程序 i 的权重值, Num_{fail} 表示错误程序的数量。

综合考虑上述两个指标, 本文采用加权求和方式整合代码覆盖率和错误检测率, 最终的适应度函数为:

$$Fit_{dec} = \alpha \times Fit_{Fcd} + \beta \times Fit_{Cov} \quad (4-5)$$

其中, α 和 β 是权重系数, 用于平衡两种评估指标的相对重要性。本文参数设置为 $\alpha = 1.0$, $\beta = 0.2$, 该组合反映了错误检测率相对于代码覆盖率的优先性, 同时保持对覆盖多样性的适当重视。

测试规模: 测试规模是评估测试数据质量的关键维度, 主要指测试数据的大小和复杂程度。OJ 系统中一个测试数据通常包含多行输入, 每行又可能包含多个测试元素。例如, 在 1070 问题中, 测试数据由两行输入组成, 每行最多包含 20000 个整数。虽然大规模测试数据能够有效检验程序在边界条件下的正确性, 但过多的大规模测试数据会降低其教学价值——学生仅知道程序在某些极端情况下失败, 却难以精确定位和复现错误, 不利于修复错误。

第三章的实验结果表明, 当前 OJ 系统上的大多数题目都过度依赖大规模测试数据。因此, 本文提出测试规模 (Test Scale, TS) 指标, 旨在保证测试覆盖率和错误检测能力的前提下, 最小化测试数据的复杂程度。

具体来说, TS 指标针对 3.2.2 节划分的数据类题目和数值类题目使用了不同的处理策略。对于数据类题目, 本文以空格为分割, 统计每一个测试输入的元素数量, 并计算其平均值与方差。为了使平均值和方差均分布在 $[0, 1]$ 区间, TS 将其分别除以了理论最大值。计算公式如下所示:

$$TS_{ave} = \frac{\sum_{i=1}^N Ele_i}{Num_{test} * Ele_{max}}, \quad TS_{var} = \frac{\sum_{i=1}^N (Ele_i - Ele_{ave})^2}{Num_{test} * Var_{max}} \quad (4-6)$$

其中, Ele_i 表示第 i 个测试输入的元素数量, Num_{test} 表示测试输入的个数, Ele_{max} 表示测试输入中最多的元素数量, Ele_{ave} 表示平均每个测试输入的元素数量, Var_{max} 表示方差的最大值。对于数值类题目, 本文统计了每个测试输入中所有整数的指数和, 并计算其平均值与方差, 计算公式如下所示:

$$TS_{ave} = \frac{\sum_{i=1}^N Exp_i}{Num_{test} * Exp_{max}}, TS_{var} = \frac{\sum_{i=1}^N (Exp_i - Exp_{ave})^2}{Num_{test} * Var_{max}} \quad (4-7)$$

其中, Exp_i 表示第 i 个测试输入的指数和, Exp_{max} 表示测试输入中最大的指数和, Exp_{ave} 表示平均每个测试输入的指数和。

TS_{ave} 从元素个数或数值大小的规模衡量了测试输入的规模, 而 TS_{var} 则反映了不同测试输入的离散程序。最终的适应度函数由两个指标加权求和得到。

$$Fit_{scale} = 1.0 \times TS_{ave} - 0.2 \times TS_{var} \quad (4-8)$$

元素数量越少或数值越低, 其可读性越好, 更方便学生复现问题进而寻找错误。 TS_{ave} 可以更直观、明确的反应测试规模。因此, 本文给予其更高的权重, 为 1.0。与常规思维相悖的是, 而当两组测试输入的元素数量或数值相同时, 本文认为方差值更大的测试数据集质量更高。这是因为 OJ 系统中算法题目通常存在明确的上下边界, 而由于输入范围极广, 上下边界的测试输入差异显著, 从而导致方差较大。优质的测试数据集应在输入范围内呈现均匀分布, 具有更高的方差; 而方差较小的测试输入往往集中分布在某一范围内, 未能充分覆盖输入空间。最终, TS_{var} 的权重被设置为-0.2。

(3) 协同演化

在完成个体适应度评估后, LLM-MOGA 算法进入协同演化阶段。本文采用两种成熟的多目标优化算法——SPEA-II 和 NSGA-II, 这些算法的介绍已在 2.2 节中阐述。

LLM-MOGA 在每代演化中应用了选择算子、变异算子和交叉算子, 通过系统性步骤生成适应度更高的新种群。在选择算子上, 算法采用锦标赛选择策略, 从当前种群中随机选取两个个体进行适应度比较, 保留适应度较高者。此过程重复执行直至筛选出与原种群规模相当的中间样本集。为保持原始种群的完整性并避免在后续操作中对原个体造成污染, LLM-MOGA 对选择得到的每个个体执行克隆操作, 生成信息相同但相互独立的副本。

随后, 算法按照预设的交叉率和变异率, 对样本种群中的个体及其相邻个体执行交叉和变异操作, 以产生多样的测试数据。由于克隆个体保留了亲本的适应度值, 操作完成后需重新计算每个个体的适应度。在对样本种群中的全部个体完成交叉和变异操作后, LLM-MOGA 将旧种群与样本种群合并, 通过所选多目标优化算法的非支配排序机制, 筛选出与原种群规模相同的优质个体作为新一代种群, 实现种群质量的稳步提升。

在演化过程中, 测试数据的质量评估主要基于错误检测率、代码覆盖率和测试规

模等关键指标。然而，OJ 系统中的算法题目通常存在着大量错误程序，若将所有错误程序纳入训练集会显著增加计算开销，导致演化过程极其缓慢，迭代效率低下。同时，对固定错误程序集的过度优化可能导致生成的测试数据对新提交的错误程序泛化能力不足，出现过拟合现象。

算法 2 LLM-MOGA 算法：

输入：FP：错误程序集；N：训练集中的基础程序数；M：种群中个体的数量；K：一个个体中测试数据的数量；T：调整数据集的迭代次数

```

24. TP ← Random_Select(FP, N)
25. population ← []
26. for i in range (M) do
27.     individual ← []
28.     for j in range (K) do
29.         testCase ← Build_Data_Random_by_Template (K)
30.         individual append testCase
31.     end for
32.     population append individual
33. end for
34. # Start Search-Based Algorithm
35. for i in range (maxIterationNumber) do
36.     newPopulation ← Select_By_Tournament (population)
37.     newPopulation ← Mutated_And_Crossover_From_newPopulation (newPopulation)
38.     FL ← Calculate_Fault_Localization_Accuracy (newPopulation, TP)
39.     CCC ← Calculate_Code_Coverage_Category (newPopulation, TP)
40.     newPopulation ← Zip (newPopulation, FL, CCC)
41.     population ← population + newPopulation
42.     population ← Filter (population, M)
43.     if i % T == 0 then
44.         FLA ← Calculate_Fault_Localization_Accuracy (population, FP)
45.         bestFLIndividual ← SelectFL_MAX_Individual (FLA)
46.         tempDetections ← Fault_Code_Detection (FP, bestFLIndividual)
47.         appendPrograms ← []
48.         for item in tempDetections do
49.             if item.result != 'Wrong' then
50.                 appendPrograms append item.program
51.             end if
52.         end for
53.         appendPrograms ← Random_Select(appendPrograms, N)
54.         TP append appendPrograms
55.     end if
56. end for
57. FLA ← Calculate_Fault_Localization_Accuracy (population, FP) # Evaluation and Output
58. FCD ← sizeof (Fault_Code_Detection(FP, population))
59. quality ← Calculate_Quality (population, FLA, FCD)
60. finalResult ← Select_Max_Quality_Individual (population, Quality)
61. return finalResult

```

针对这一挑战，LLM-MOGA 采用了动态训练集构建策略。算法启动时，从错误程序库中随机选择 20 个程序构成初始训练集，为演化提供基础评估环境。随着演化进程，LLM-MOGA 实施增量更新机制：每完成 100 次迭代，算法使用当前最优个体对全部错误程序进行一次全面评估，从未被检测出的错误程序中随机选择 20 个加入

训练集，实现训练集的动态扩充。这种策略既保证了演化效率，又能持续引入新的错误程序，增强生成测试数据的泛化能力。

算法 2 展示了 LLM-MOGA 的完整演化流程，该算法的输入包括所有错误程序（FP）、训练集中的基础程序数(N)、种群中的个体数(M)、个体中的测试数据数(K)和调整训练集的迭代次数(T)。执行流程中，算法首先初始化训练集和种群(第 1-10 行)，然后在迭代中应用遗传算子生成新种群并计算新个体的适应度(第 14-18 行)。接下来再通过多目标优化算法从旧种群与样本种群中筛选出新种群（第 19-20 行）。每经过 T 代迭代，算法使用当前最优个体对全部错误程序进行评估，并根据结果动态更新训练集(第 23-33 行)。最终，算法输出满足多目标优化的高质量测试数据集，该测试数据集在错误检测能和测试规模两个维度上实现了平衡优化。

（4）最终评估

经过 500 次迭代或当种群在 100 代进化中适应度均保持不变时，LLM-MOGA 会结束搜索的过程。此时，LLM-MOGA 会在完整的数据集（包含全部错误程序和假阳性程序）上计算每个个体的错误检测率及测试规模，综合评估其性能并选择最终解。

4.3 实验设计

4.3.1 研究问题

为了评估 mftdg 生成测试数据的有效性，本章节提出以下三个研究问题：

研究问题 1：与基准方法相比，LLM-MOGA 方法生成的测试数据集的有效性如何？

本问题旨在全面评估 LLM-MOGA 方法与基准方法的对比优势。本章节将从代码覆盖率、错误检测能力和测试规模等多个维度进行比较分析。通过这些关键指标的对比，可以客观评估 LLM-MOGA 方法在生成高质量测试数据方面的实际表现。

研究问题 2：适应度函数中不同的参数组合对 LLM-MOGA 的性能有何影响？

本问题旨在探究 LLM-MOGA 方法中适应度函数参数的权重配置分析。本章节将系统地调整各项参数，观察它们对测试数据生成质量和效率的影响。这种权重分析不仅有助于找到 LLM-MOGA 方法的最优配置，还能深入理解多目标优化算法中各因素的相对重要性。

研究问题 3：大语言模型与多目标优化算法分别对 LLM-MOGA 的贡献如何？

本问题旨在解构 LLM-MOGA 方法的核心组件，分别评估 LLM 和多目标优化算法对整体性能的贡献度。通过控制变量法，本文将分别移除这两个关键组件，观察系统性能变化。

4.3.2 评价指标

第三章介绍的评价指标外，为了便于对比不同方法生成测试数据的测试规模，本章节提出了一种测试规模的度量指标 TC。本文基于 3.3.2 的题目分类，对不同类型的题目采用了不同的计算方法。数据类题目的计算公式为：

$$TC = \frac{\sum_{i=1}^N Num_i}{Top_{input}} \quad (4-10)$$

其中， N 表示测试数据的个数， Num_i 表示第 i 个测试数据的测试元素数， Top_{input} 表示一个测试输入中测试元素的数量上限。数值类题目的计算公式为：

$$TC = \frac{\sum_{i=1}^N Sum_i}{Top_{input}} \quad (4-11)$$

其中， Sum_i 表示测试输入中所有元素的指数和， Top_{input} 表示一个测试输入中测试元素的指数上限。在 OJ 系统中，算法题目通常会给定一个输入上限从而构建题目边界。在多数情况下，题目只需构造一个达到该上限的测试数据即可有效验证学生程序在边界条件下的正确性，而常规测试数据则无需达到同等规模。因此，理想的测试数据集的 TC 值应该略高于 1。

4.3.3 基准方法

本文与人工设计的测试数据、Zhou^[74]等人使用的遗传算法以及 Liu^[75]等人提出的 AID 方法进行了对比。Zhou 等人提出了一种错误定位引导的测试数据生成方法，该方法利用遗传算法构建 OJ 问题的搜索算法生成测试数据，以测试数据的 EXAM 值的倒数为适应度，自动生成高质量的测试数据。AID 则结合了 LLM 与差分测试，通过 LLM 生成错误程序的修复变体及测试数据，随后通过差分测试找出可以检测假阳性程序的高质量测试数据。

4.4 结果分析

4.4.1 本文方法的有效性分析

在研究问题 1 中，本文分别使用了 SPEA-II 和 NSGA-II 两种多目标优化算法生成测试数据。实验结果如表 4-1、4-2 所示，其中第一列为题目编号，第二列表示不同的

测试数据生成方法以及使用了不同多目标优化算法的 LLM-MOGA，第三列展示了这些方法生成测试数据的错误检测率，第四、五列分别展示了行覆盖率与分支覆盖率，第六列则展示了测试数据的规模。其中，最佳实验结果被高亮显示。为了便于展示，本章节将分别统计所有方法在 BuctOJ 和 AtCoder 数据集上的实验结果。

表 4-1 BuctOJ 数据集上各方法的实验结果

Table 4-1 Experimental results of each approach on the BuctOJ dataset

题目 ID	方法	错误检测率	行覆盖率	分支覆盖率	测试规模
1069	Manual	89.38%	76.43%	84.42%	0.20
	LLM-MOGA (SPEA-II)	91.68%	78.28%	89.38%	0.22
	LLM-MOGA (NSGA-II)	97.78%	78.28%	89.38%	2.42
	GA	89.38%	73.28%	86.24%	4.28
1070	Manual	79.04%	81.11%	85.46%	0.20
	LLM-MOGA (SPEA-II)	98.12%	86.85%	87.03%	1.37
	LLM-MOGA (NSGA-II)	98.12%	86.85%	87.03%	1.22
	GA	87.08%	84.79%	86.10%	2.29
1119	Manual	97.88%	80.91%	73.14%	2.01
	LLM-MOGA (SPEA-II)	99.42%	85.16%	80.65%	3.01
	LLM-MOGA (NSGA-II)	98.85%	81.97%	76.43%	1.45
	GA	94.71%	77.62%	70.84%	4.08
2130	Manual	96.00%	83.05%	78.77%	1.05
	LLM-MOGA (SPEA-II)	97.05%	84.44%	82.22%	1.64
	LLM-MOGA (NSGA-II)	97.53%	84.44%	82.22%	1.29
	GA	93.06%	83.72%	80.64%	19.38
4254	Manual	89.63%	84.27%	75.28%	1.00
	LLM-MOGA (SPEA-II)	100.00%	82.87%	82.50%	1.00
	LLM-MOGA (NSGA-II)	100.00%	84.12%	82.50%	1.00
	GA	94.15%	85.46%	79.15%	1.00
6498	Manual	91.49%	80.10%	72.37%	13.74
	LLM-MOGA (SPEA-II)	98.08%	84.55%	76.67%	1.80
	LLM-MOGA (NSGA-II)	98.08%	84.55%	76.67%	1.80
	GA	85.49%	80.48%	78.15%	15.09

从表 4-1 可以看出，LLM-MOGA 实现了最佳的错误检测能力。具体来说，LLM-MOGA 在全部题目上都可以达到 90%以上的错误检测率，平均错误检测率达到了 98.39%，相比于现有测试数据和 GA 方法生成的测试数据分别提升 7.82%和 7.75%。同时，LLM-MOGA 也是唯一一种可以生成 100%错误检测率测试数据的方法。这是因为人工设计测试数据时难以考虑到所有的边界情况，GA 方法以测试数据的 EXAM 值

为导向，生成的测试数据具有较好的错误定位能力，而削弱了错误检测能力。

在代码覆盖率方面，LLM-MOGA 的平均行覆盖率达到 83.69%，高于现有测试数据的 80.98%和 GA 方法的 80.89%。在分支覆盖率上，LLM-MOGA 同样实现了最高的平均分支覆盖率（83.08%），相比现有测试数据和 GA 方法提升了 4.84%和 2.89%。

表 4-2 AtCoder 数据集上各方法的实验结果

Table 4-2 Experimental results of each approach on the AtCoder dataset

题目 ID	方法	错误检测率	行覆盖率	分支覆盖率	测试规模
abc364b	Manual	97.67%	85.28%	82.78%	15.33
	LLM-MOGA (SPEA-II)	98.27%	88.78%	90.42%	1.82
	LLM-MOGA (NSGA-II)	97.79%	88.78%	86.11%	1.80
	GA	94.32%	82.48%	83.15%	9.73
abc364c	Manual	97.65%	83.59%	77.15%	52.94
	LLM-MOGA (SPEA-II)	94.54%	89.46%	79.52%	3.42
	LLM-MOGA (NSGA-II)	99.02%	81.15%	79.52%	2.34
	GA	91.59%	82.45%	76.87%	3.82
abc365c	Manual	93.87%	78.71%	90.39%	225.21
	LLM-MOGA (SPEA-II)	95.79%	80.00%	92.71%	22.09
	LLM-MOGA (NSGA-II)	95.98%	80.00%	93.13%	34.27
	GA	91.58%	78.91%	89.45%	174.93
abc366b	Manual	92.53%	81.22%	80.59%	3.98
	LLM-MOGA (SPEA-II)	97.62%	88.89%	77.78%	1.72
	LLM-MOGA (NSGA-II)	98.11%	88.89%	77.78%	3.41
	GA	95.78%	83.15%	71.51%	5.95
abc366c	Manual	98.77%	80.30%	79.73%	37.42
	LLM-MOGA (SPEA-II)	99.61%	82.22%	84.78%	23.73
	LLM-MOGA (NSGA-II)	99.11%	82.98%	84.78%	1.32
	GA	90.92%	79.61%	80.24%	27.45
abc373b	Manual	74.70%	85.53%	72.16%	1.00
	LLM-MOGA (SPEA-II)	91.11%	84.57%	76.11%	1.00
	LLM-MOGA (NSGA-II)	91.11%	84.57%	76.11%	1.00
	GA	82.22%	80.89%	65.43%	1.00

在测试规模方面，LLM-MOGA 在大多数题目中生成的测试数据均具有最小的测试规模。尽管在题目 1069 和 1070 上，现有测试数据集的测试规模小于 LLM-MOGA，但其 TC 值远低于 1，仅为 0.2。过小的测试规模无法有效检测学生程序在边界情况下的正确性，从而导致其错误检测率偏低。总体而言，LLM-MOGA 生成测试数据的平均 TC 值为 1.51，而现有测试数据和 GA 方法的平均 TC 值分别为 3.04 和 7.68，分别

是 LLM-MOGA 的 2.01 倍和 5.08 倍。

表 4-2 展示了三种方法在 AtCoder 数据集上的实验效果，可以看出在难度更高的 AtCoder 数据集上，LLM-MOGA 仍然保持着最佳的实验结果。LLM-MOGA 的平均错误检测率为 96.85%，高于现有测试数据的 92.53% 和 GA 方法的 91.07%。此外，LLM-MOGA 也实现了最佳的平均行覆盖率（85.65%）和分支覆盖率（83.55%），以及最优的平均 TC 值（7.36）。

表 4-3 LLM-MOGA 与 AID 的假阳性程序检测结果

Table 4-3 False-positive program detection results of LLM-MOGA and AID

题目 ID	假阳性程序数	假阳性程序检测率	
		LLM-MOGA	AID
1069	134	97.01%	68.66%
1070	769	91.94%	100.00%
1119	22	90.91%	72.73%
2130	76	90.79%	60.53%
4254	62	100.00%	51.61%
6498	71	77.46%	83.10%
abc364b	39	79.49%	74.36%
abc364c	43	83.72%	27.91%
abc365c	131	48.85%	28.24%
abc366b	154	88.96%	91.56%
abc366c	25	76.00%	80.00%
abc373b	333	96.40%	49.55%

在假阳性程序检测方面，本章节将 LLM-MOGA 与 AID 方法进行了对比，实验结果如表 4-3 所示。结果显示，在大多数题目中，LLM-MOGA 的假阳性程序检测率均高于 AID。LLM-MOGA 的平均检测率达到 85.13%，而 AID 的平均检测率仅为 65.69%。尤其是在输入元素较多的题目（如问题 abc364c、2130 等）上，LLM-MOGA 与 AID 的性能差距尤为显著。为了深入研究 AID 效果不佳的原因，本章节手动分析了 AID 的执行过程。结果表明，AID 的实验效果受到 LLM 的程序修复能力和代码生成能力的影响。其效果不佳主要有两个原因：（1）LLM 未能正确修复假阳性程序，生成正确的程序变体。（2）即使 AID 生成了正确的程序变体，但基于 LLM 生成的测试数据质量有限，未能通过生成与原程序产生差异输出的高质量测试数据。

综上所述，LLM-MOGA 可以生成具有高效错误检测能力的测试数据，同时兼顾测试数据的测试规模。为了直观展示多目标优化过程中各目标之间的权衡关系，本文以题目 1069 为例，绘制了优化过程中的帕累托前沿，如图 4-2 所示。

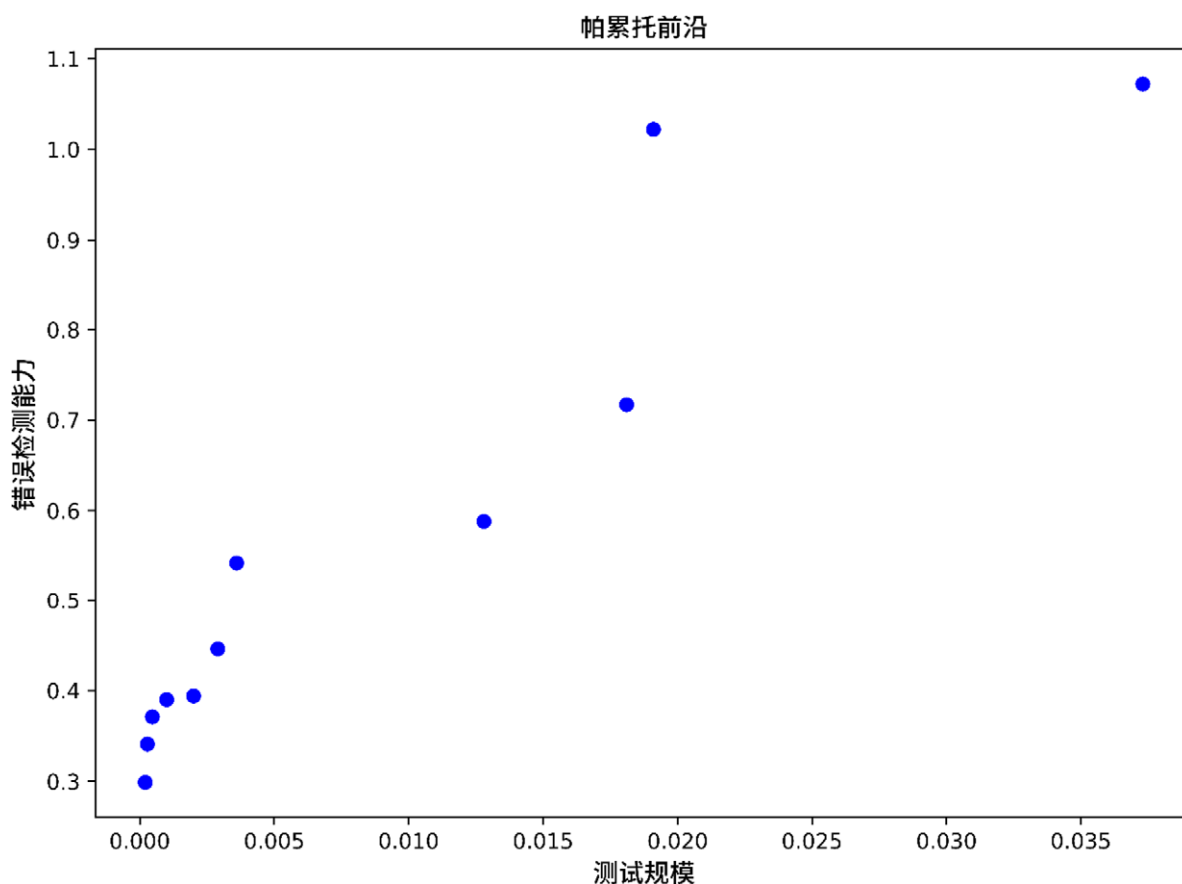


图 4-2 帕累托前沿

Fig.4-2 Pareto Front

从图中可以看出，随着测试数据规模的减小，错误检测能力出现不同程度的下降。这一趋势验证了两个目标之间存在天然的冲突：规模较小的测试数据更易于学生理解和复现错误，但其错误检测能力较弱；而规模较大的测试数据虽然能提升检测能力，但却降低了可读性和教学辅助效果。

4.4.2 本文方法参数权重配置

为探究 LLM-MOGA 中两个适应度函数的权重配置对测试数据生成效果的影响，本文设计了 9 组不同的权重参数组合，并收集了在这些参数组合下的实验结果。参数配置基于以下原则：

错误检测率：作为测试数据的首要目标，错误检测率在错误检测能力的评价中应占据主导地位。本实验为其设置了 1.1 至 0.7 的权重区间，体现其核心地位。

代码覆盖率：作为衡量测试数据检测范围的重要指标，代码覆盖率能间接提升错误检测能力。但考虑到其辅助性质，本实验为其配置了适中的权重区间（0.2 至 0.4）。

平均元素数：在 OJ 场景下，简短的测试数据对学生的教育反馈价值显著，平均元素数直观体现了测试数据的规模。本实验为其设置了 0.9 至 1.1 的权重区间。

元素数方差：用于衡量一组测试数据集中不同类型测试数据的分布情况。在平均元素数量相同的前提下，元素数的方差越大，测试的覆盖范围越广，因而测试的完备性更强。本实验为其设置了-0.1至-0.3的权重区间。

表 4-4 详细列出了最终的 9 组权重组合方案。

表 4-4 LLM-MOGA 个体适应度权重组合

Table 4-4 Individual fitness weight combinations in LLM-MOGA

序号	错误检测率	代码覆盖率	平均元素数	元素数方差
1	1.1	0.2	0.9	-0.3
2	1	0.3	0.9	-0.3
3	0.9	0.4	0.9	-0.3
4	1	0.2	1.0	-0.2
5	0.9	0.3	1.0	-0.2
6	0.8	0.4	1.0	-0.2
7	0.9	0.2	1.1	-0.1
8	0.8	0.3	1.1	-0.1
9	0.7	0.4	1.1	-0.1

初步实验表明，在不同算法题目，以及相同算法题目下不同算法变体的输出结果存在相似性。因此，本文主要聚焦于分析 LLM-MOGA（SPEA-II 实现）在问题 1070 上的表现。表 4-5 呈现了在 9 组不同参数组合下的实验数据。

表 4-5 不同参数组合下的实验效果

Table 4-5 Experimental results under different parameter combinations

序号	错误检测率	行覆盖率	分支覆盖率	测试规模
1	98.12%	86.85%	87.03%	2.90
2	93.00%	87.96%	87.03%	1.05
3	96.85%	86.85%	88.38%	1.25
4	98.12%	86.85%	87.03%	1.22
5	93.33%	86.85%	87.03%	0.89
6	96.70%	89.98%	87.03%	1.12
7	93.96%	86.85%	87.03%	0.96
8	92.94%	88.22%	87.03%	1.19
9	85.33%	90.07%	88.38%	0.73

表 4-5 中的实验数据揭示了测试数据的错误检测能力与测试规模之间存在的正相关关系。具体而言，测试规模较大的用例（如组合 1 的 2.90）能够更加全面地覆盖程

序的边界情况和特殊场景,从而更有效地暴露学生程序中潜在的逻辑缺陷和实现错误。并且,此类测试数据能够更准确地评估学生程序所采用算法的时间和空间复杂度,进而识别出逻辑正确但性能低下的实现方案。因此,在其他条件保持相对稳定的前提下,适度增加测试规模可以提升测试数据的错误检测能力。

此外,从实验结果可以看出,在 LLM-MOGA 中测试数据的代码覆盖率的提升并不必然导致错误检测能力的增强。对比参数组合 1、3 可以看出,组合 3 的代码覆盖率权重最高,而其最终的错误检测率却弱于组合 1。但同样不可以忽略其在错误检测能力中的贡献,例如对比参数组合 5、6 可以看出,尽管组合 5 错误检测率的权重更高,但其最终错误检测率却低于组合 6。因此,可以得出以下结论:代码覆盖率可以引导种群覆盖更多的代码路径,提高其检测范围进而提升错误检测能力。但过度追求高覆盖率可能导致算法将有限资源分散投入到各种程序路径上,包括不易触发错误或错误出现概率极低的路径,从而稀释了测试数据在关键路径上的检测能力。

通过对实验数据的综合分析,参数组合 4(错误检测率权重 1.0,代码覆盖率权重 0.2,测试规模权重-1.0)展现出最佳的综合性能。该组合实现了与组合 1 相同的高错误检测率,同时将测试规模控制在适当的值。因此,本文最终选择了组合 4 作为本实验的参数配置。

4.4.3 不同模块对本文方法的贡献

LLM-MOGA 通过 LLM 与多目标优化算法这两大核心模块的协同作用,显著提高了测试数据的生成质量。在这一方法中,LLM 负责生成初始测试数据,提升初始种群多样性与自动化程度;而多目标优化算法则基于这些初始用例不断迭代优化,最终生成兼顾错误检测能力及测试规模的高质量测试数据集。

为深入探究这两大核心模块各自的贡献及其协同效应,本文设计了系统性消融实验。通过分析各组件单独作用时的效果,本文可以更准确地评估完整方法中各模块的价值。具体而言,本章节设计了两组实验配置:仅使用 LLM 直接生成测试数据;采用随机算法生成初始种群,随后应用多目标优化算法进行迭代优化。这两组实验的结果将与完整 LLM-MOGA 方法进行对比,从而量化各组件的贡献度。

表 4-6 展示了仅使用 LLM 生成的测试数据在各指标上的表现。实验数据显示,LLM 单独作用时生成的测试数据表现出多方面的不足。大模型生成的测试数据错误检测率呈现高度波动性,从最低的 49.88%(1069)到最高的 87.53%(6498)不等,平均错误检测率为 75.44%。这一数值显著低于完整 LLM-MOGA 方法的平均错误检测率。因此,LLM 虽然能基于题目描述生成相关测试数据,但其生成策略主要依赖于对问题语义的理解,难以系统性地覆盖程序边界情况和异常路径。

表 4-6 大语言模型生成测试数据质量分析

Table 4-6 Quality analysis of test data generated by LLM

题目编号	错误检测率	行覆盖率	分支覆盖率	测试规模
1069	49.88%	78.28%	59.38%	3.00
1070	83.43%	80.85%	75.68%	4.00
1119	68.08%	69.80%	60.71%	25.00
2130	81.60%	74.44%	75.00%	25.00
4254	86.05%	70.84%	89.51%	1.00
6498	87.53%	84.55%	66.67%	7.00
abc364b	84.61%	70.28%	67.86%	10.50
abc364c	61.67%	76.51%	90.00%	15.00
abc365c	65.71%	74.19%	73.08%	14.00
abc366b	85.09%	68.89%	77.78%	10.50
abc366c	69.15%	83.33%	86.84%	12.50
abc373b	82.44%	78.39%	62.50%	1.00

在代码覆盖率方面，LLM 生成的测试数据平均行覆盖率为 72.46%，分支覆盖率为 75.40%。由于许多错误往往隐藏在较少被执行到的边界路径中，代码覆盖率不足直接导致了错误检测能力的下降。这表明 LLM 生成的测试数据对程序内部结构的感知能力有限，难以精准覆盖关键执行路径。

测试规模是另一个突出问题。数据显示，LLM 生成测试数据的测试规模普遍偏高，与 LLM-MOGA 方法相比增加了 2.41 倍。这一现象与本文使用的均匀生成策略有关，尽管该策略生成的测试数据相比随机生成具有更好的覆盖情况，但也造成了大量的测试冗余。这种冗余不仅降低了测试效率，还增加了测试的执行和维护成本。

表 4-7 展示了仅使用多目标优化算法生成的测试数据的表现，可以看出多目标优化算法单独作用时生成的测试数据质量相比于 LLM 单独作用有了显著提升，平均错误检测率从 75.44% 提高到 90.14%，提升了 14.7%，平均行覆盖率提升了 1.43%，分支覆盖率提升了 1.87%，并且具有更小的测试规模。这证明了多目标优化算法在测试数据生成领域的有效性，通过迭代优化能够逐步提高测试数据的质量。

然而，多目标优化算法单独作用时仍存在部分缺陷。首先，在错误检测能力方面，其平均错误检测率相比完整的 LLM-MOGA 仍有 7.49% 的差距，并且在部分问题表现不佳。如在 2130 题目上错误检测率仅为 76.67%，远低于平均水平。通过手动分析发现，这是由于随机算法生成的初始种群多样性不足，测试输入集中在特定区间内，导致多目标优化算法陷入局部最优解，并在后续迭代中未能探索到更广阔的解空间。其次，在代码覆盖率方面和测试规模方面，虽然多目标优化算法生成的测试数据的代码覆盖率和测试规模优于 LLM 单独作用时的表现，但几乎全面落后于 LLM-MOGA。这

表明初始种群的质量对多目标优化算法的搜索过程具有较大影响。

表 4-7 多目标优化算法生成测试数据质量分析

Table 4-7 Quality analysis of test data generated by MOOA

题目编号	错误检测率	行覆盖率	分支覆盖率	测试规模
1069	84.50%	71.38%	62.50%	1.62
1070	72.51%	80.85%	75.68%	2.93
1119	94.35%	79.80%	74.29%	0.97
2130	90.50%	76.67%	69.44%	10.23
4254	99.83%	80.00%	82.5%	1.00
6498	92.29%	84.55%	66.67%	0.83
abc364b	93.07%	78.78%	82.78%	2.58
abc364c	92.00%	76.00%	79.52%	40.53
abc365c	89.88%	80.00%	90.39%	27.89
abc366b	96.31%	68.89%	77.78%	2.41
abc366c	95.15%	72.00%	84.78%	0.82
abc373b	81.25%	78.57%	61.11%	1.00

综合上述内容，可以看出 LLM 生成的高质量测试数据为多目标优化算法提供了多样的初始种群，提高了其搜索空间，进而影响最终的测试数据质量。并且仅需修改题目描述信息就可以针对不同的题目生成符合题目约束的测试数据生成器，大幅提高了 LLM-MOGA 方法的自动化程度。而多目标优化算法通过迭代优化提高了测试数据的错误检测能力，并降低了其测试规模。证明了两个模块的协同效应和必要性。

4.5 本章小结

本章节从第三章的实验结果出发，提出了一种兼顾错误检测能力和测试规模的测试数据自动生成方法 LLM-MOGA。该方法通过提示词工程引导 LLM 生成具有多样性的初始种群，随后通过多目标优化算法进行迭代优化，最终生成高质量测试数据。在研究问题 1 中，LLM-MOGA 生成的测试数据与其他方法相比取得了最佳的错误检测能力和代码覆盖率，并在大多数题目上具有更小的测试规模。在研究问题 2 中，本文研究了不同的权重参数对测试数据生成效果的影响。本实验设置了多种参数组合，并分析不同组合的实验效果，最终选取了最佳的参数组合。研究问题 3 探讨了 LLM 和多目标优化算法分别对 LLM-MOGA 的影响，通过评估各模块单独作用时的实验结果验证了两个模块的互补性与协同作用。

第五章 面向在线评测系统的测试数据生成与评估工具原型设计

本章结合本文提出的测试数据评估方法及测试数据生成方法，设计并实现了其工具原型（Test data Evaluate and Generate, TestEG）。TestEG 旨在帮助 OJ 系统管理员分析现有的测试数据质量，并自动生成高质量测试数据。

5.1 系统概述

TestEG 主要分为前端界面、后端处理与数据库存储三个部分，其系统流程如图 5-1 所示。

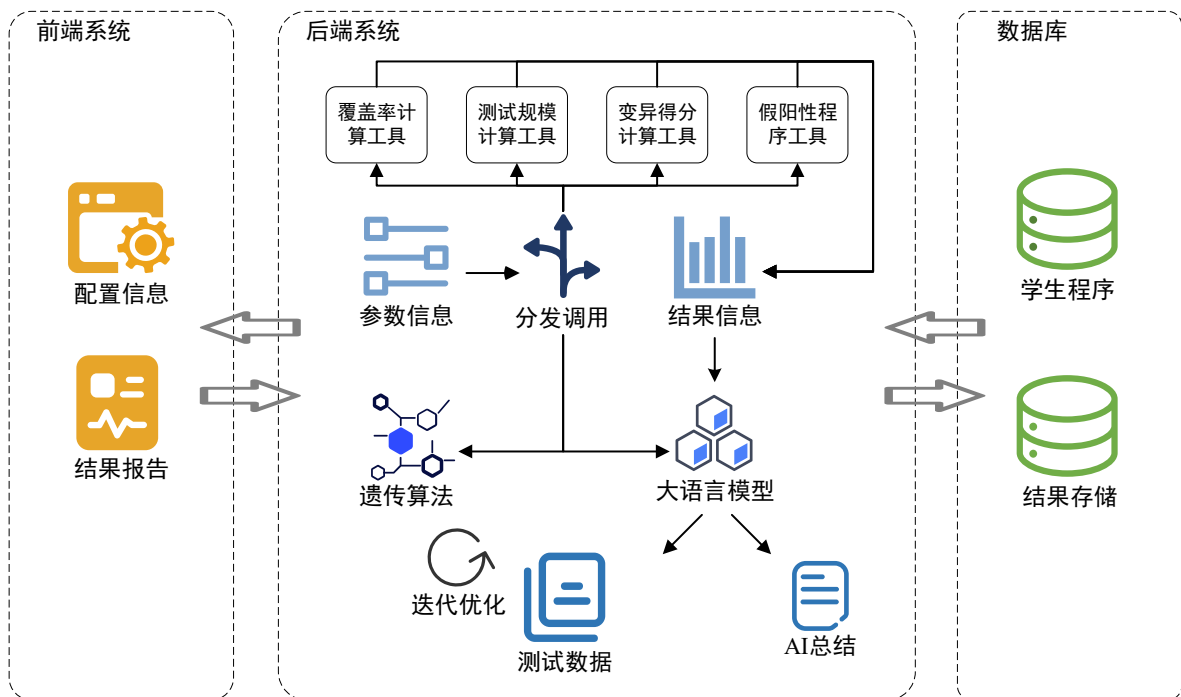


图 5-1 TestEG 系统流程图

Fig.5-1 Flowchart of the TestEG

在前端界面，管理员可以在题目页面选择评估该题目的测试数据质量或生成高质量测试数据。在评估测试数据时，管理员可以从代码覆盖率、变异得分、假阳性程序和测试规模四个指标出发，自定义算法参数，对现有的测试数据质量进行评估。最终结果以表格、柱状图等形式展示，并有 AI 基于结果内容给出分析和意见。用户也可以在管理界面查看已经评估完成的评估报告。在生成测试数据时，用户可以选择仅使用 LLM 快速生成符合题目约束的测试数据，或使用多目标优化算法对测试数据进行迭代优化，生成高质量测试数据。两种方式都提供了丰富的参数设置，让用户可以基于自身需求调整测试数据的生成方式。测试数据生成完毕后将以文本的形式展示在网

页中。

后端主要负责解析前端发送的请求参数，创建评估、生成任务并存储在数据库中。对于评估任务，后端调用其相应的检测工具并从数据库中拉取数据集。在评估完成后将结果信息存储在数据库中。对于生成任务，后端则调用 LLM API 或多目标优化算 0 法工具，并将生成的测试数据以文件的形式存储在服务器中。

数据库则负责存储学生程序、所有的评估和生成任务、任务执行结果以及部分任务执行时的中间数据。

除上述功能外，TestEG 的设计还考虑了以下的非功能性需求，确保系统的质量和可靠性：

（1）易用性：用户界面设计需要清晰简洁，操作流程应当直观且符合用户的常规使用习惯。界面设计应帮助用户快速理解系统的工作方式，降低学习门槛。

（2）可靠性：系统拥有自动化备份能力，在面临诸如断电或网络故障等突发情况时，具备自动保护和数据恢复的机制，防止数据的丢失或损坏。

（3）可扩展性：由于业务需求可能在未来发生变化，系统应当可以轻松适应这些变化，支持功能上的迭代更新。在设计阶段，模块化和低耦合的设计原则能够帮助系统更灵活地应对变更，同时降低维护的复杂度。

5.2 系统架构设计

TestEG 采用 B/S 架构实现，其架构设计包含表现层、业务逻辑层和数据访问层。图 5-2 展示了 TestEG 的系统架构图。

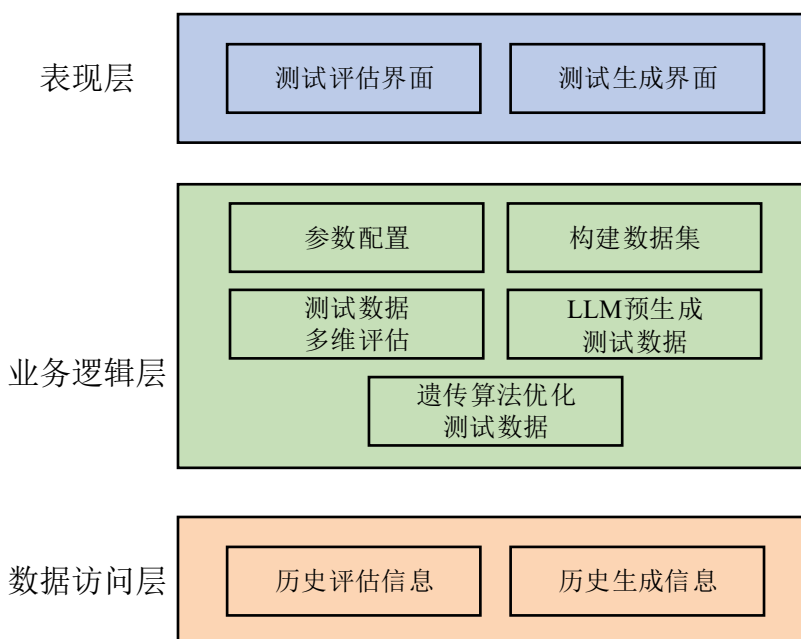


图 5-2 系统架构图

Fig.5-2 System architecture diagram

其中，表现层是系统的入口，它负责处理用户的交互需求。此层接收用户请求，并展示测试数据的评估结果与生成结果。

业务逻辑层负责整个系统的核心业务流程。该层读取用户输入并进行处理，执行一系列逻辑任务，包括参数配置、构建数据集、多维评估和测试生成等，任务执行结果通过表现层进行可视化展示。

数据访问层的任务是管理系统的数据存储与检索。它根据用户提交的信息进行数据库读取及写入，记录操作过程以确保数据的完整性与一致性。

5.3 核心模块设计

本章节详细介绍了 TestEG 的核心模块。TestEG 主要由测试数据评估和测试数据生成两大模块组成。这两大模块构成了 TestEG 的基础框架，实现了其功能的完整性。

5.3.1 测试数据评估

测试数据评估模块旨在从代码覆盖率、变异得分、测试规模和假阳性程序四个维度对现有测试数据进行全面评估。该模块采用统一的工程架构设计，包含配置管理、数据集构建、工具调用和结果存储四个层次，通过标准化的流程和接口实现评估工作的自动化与可扩展性。各层次的设计与实现如下所述：

（1）配置管理层

配置管理层负责为不同评估维度提供参数定制能力，针对不同评估维度的特性实现了差异化的配置策略，同时保持了统一的处理框架。

在代码覆盖率维度，TestEG 设计了三类关键配置参数：覆盖指标选择、目标程序选择以及测试范围选择。这些配置参数使管理员能够根据不同程序设计题目的特点调整评估策略。

在变异得分维度，TestEG 提供了测试范围选择及变异算子选择两类配置参数。在变异算子选择方面，系统支持针对程序特征进行个性化配置，如对计算密集型程序偏重使用算术变异算子，对控制流复杂程序增加逻辑变异算子比例，实现了评估方法的适应性调整。

在测试规模维度，TestEG 实现了测试指标选择，使管理员能够根据题目的输入特性和复杂度，选择合适的规模量化标准。

在假阳性程序检测维度，TestEG 设计了聚类开关和检测算法选择参数，管理员可以根据数据库中的程序数量及题目难度调整检测策略。

所有配置信息统一转换为 JSON 格式文件存储，实现了配置数据的结构化表达和标准化处理，为系统的模块解耦和功能扩展提供支持。

（2）数据集构建层

数据集构建层负责将配置信息转化为实际的待测数据集，实现了配置解析、数据查询和本地存储三个处理阶段。

首先，数据集构建层解析 JSON 配置文件，提取关键参数并转换为结构化的数据库查询语句。该环节采用参数化 SQL 查询技术，有效防止 SQL 注入，同时提高了查询效率。随后，后端系统执行这些查询语句，从数据库中精准检索符合条件的学生程序，并将结果集写入本地文件系统。为优化磁盘空间使用，系统实现了增量更新机制，避免重复下载已有程序。

此外，数据集构建层还根据题目 ID 自动拉取相应的测试数据集，保存为 txt 格式文件。为确保后续工具正确识别和处理这些文件，本系统设计了专用的 Python 脚本统一调整测试文件的命名规范和目录结构。通过这一系列处理，TestEG 实现了从抽象配置到具体测试对象的转换，为各评估维度提供了数据基础。

（3）工具调用层

工具调用层是测试数据评估模块的核心执行环节，负责调用相应的评估工具完成实际分析任务。该层采用了适配器设计模式，统一不同工具的调用接口，提高了系统的可维护性和可扩展性。

在代码覆盖率维度，TestEG 实现了基于编译插装的覆盖率分析方法。工具首先在编译阶段对目标程序插入覆盖率探测点，然后执行测试数据并生成覆盖信息记录文件，最后通过 Set 集合数据结构对覆盖信息进行去重统计，计算覆盖率指标。为应对执行异常，系统设计了超时控制和异常处理机制，确保评估过程的稳定性。

在变异得分维度，TestEG 实现了变异体生成与测试执行工具。该工具首先通过语法分析识别代码中的潜在变异点，然后根据配置的变异算子生成变异体，并对其进行编译验证以过滤无效变异体。在执行阶段，TestEG 运行测试数据检验各变异体，记录其行为特征，并计算最终的变异得分。

在测试规模维度，TestEG 实现了测试规模分析工具。该工具将测试数据分解为最小粒度的测试元素，并基于配置参数进行多维度统计，计算综合 TC 值作为规模评估指标。

在假阳性程序检测维度，TestEG 结合了 DFC 和 SFDT 两种技术。收集所有正确程序的动态特征并进行聚类，并基于程序特征和题目信息生成针对性测试数据。基于聚类结果和生成的测试数据，通过差分测试方法检测潜在的假阳性程序。

工具调用层的设计既确保了各评估工具的独立性，又实现了调用接口的标准化，为系统的功能扩展提供了良好支持。

（4）结果存储层

结果存储层负责统一管理各评估维度的计算结果和中间数据，实现了评估数据的持久化存储和高效检索。

该层设计了标准化的数据库表结构，包括评估配置表、执行记录表和结果数据表三个主要部分。评估配置表存储每次评估的参数设置；执行记录表记录评估过程的时间戳、执行状态等元信息；结果数据表则分维度存储具体的评估指标。这种结构设计既保证了数据的完整性，又支持多维度的查询分析。

通过以上四个层次的协同工作，测试数据评估模块实现了从配置设定到结果生成的完整评估流程。

5.3.2 测试数据生成

测试数据生成模块基于创新的 LLM-MOGA 方法实现高质量测试数据的自动化生成。该模块结合了大型语言模型的语义理解能力和多目标优化算法的调优能力，形成了“生成-优化-评估”的生成体系。本模块采用四层架构设计，包含配置管理层、测试数据预生成、多目标优化和质量评估四个层次，详细介绍如下所述：

（1）配置管理层

配置管理层负责 LLM 调用和多目标优化算法的参数配置。

在 LLM 参数配置方面，TestEG 设计了包括温度系数、最大 token 数等关键参数的配置接口，这些参数直接影响 LLM 生成测试数据的多样性和创造性。管理员可以基于生成需求和题目复杂度来调整这些参数，实现了生成过程的控制。

在多目标优化算法配置方面，TestEG 提供了包括变异率、交叉率、适应度权重等多个参数配置选项。管理员可以根据测试数据的生成目标和生成效率灵活调整各参数的权重，引导优化算法向期望的方向进行测试数据优化。

所有配置信息以统一的 JSON 格式存储，实现了配置的标准化存储，为后续的模型迭代和优化策略调整提供了便捷支持。

（2）预生成层

预生成层负责与 LLM 交互并生成初始测试数据集，是整个生成流程的起点。TestEG 解析配置管理层生成的 JSON 文件，获取 LLM 参数设置。随后从题目库中提取目标题目的描述信息、输入输出规范和示例。并将这些信息与预设的提示词模板进行智能组合，形成完整的提示输入。

随后，TestEG 通过 API 接口向大型语言模型发送请求，获取测试数据生成器。这一步采用了请求重试和异常处理机制，确保在网络波动或模型服务不稳定的情况下仍能可靠获取结果。TestEG 会对生成的代码进行编译测试和标程验证，确保生成器的正确性。

最后，TestEG 执行该测试数据生成器，生成初始测试输入，并通过标程获取输出，组合形成完整的测试数据集。通过这一系列处理，预生成层实现了从题目语义到初始测试数据集的自动生成，为后续的多目标优化过程提供了多样化的候选解集。

（3）多目标优化层

多目标优化层负责对预生成的测试数据进行进一步优化，提升其质量和有效性。TestEG 首先读取配置管理层中的多目标优化算法参数，包括种群规模、迭代次数、交叉概率、变异概率等关键控制参数，并将其转换为标准化的 `config` 配置文件。将算法参数与算法工具解耦，便于算法调整 and 性能调优。

随后，系统调用 MOGA 工具。该工具首先进行内部初始化，加载配置参数。MOGA 将预生成层输出的测试数据文件读入内存，经过解析和编码处理后构建为初始种群，并开始迭代进化。

每一轮迭代中，MOGA 首先对种群中的个体多维度评估，计算其在管理员所选指标上的适应度值。接下来在种群的克隆样本上通过交叉算子和变异算子生成新的个体，其中交叉操作能够有效组合不同测试数据的优势，而变异操作则增加了种群多样性，防止算法陷入局部最优解。然后基于 SPEA-II 或 NSGA-II 算法筛选出新种群。

经过预设的迭代次数或达到收敛条件后，MOGA 工具从最终种群中选择最优个体作为搜索结果。

（4）结果存储层

该层设计了三类核心数据的存储方案：LLM 预生成的初始测试数据集、MOGA 优化过程中的中间状态数据以及最终优化后的测试数据集。对于初始测试数据和最终测试数据，TestEG 采用文本格式（.txt 文件）存储在服务器的特定目录中，并通过规范的命名约定区分不同题目、不同生成批次的测试数据。这种设计便于测试数据的查看和手动分析。

对于优化过程中的中间状态数据，TestEG 采用了结构化的日志文件进行存储。中间状态数据包含每一代种群的适应度值、执行时间以及个体的基因编码，这些数据对于分析算法性能、调优优化参数具有重要价值。

为方便后续的数据检索和分析，TestEG 建立了层次化的目录结构。每次生成和优化任务都被分配唯一的 ID，所有相关数据被存储在对应的会话目录下，同时 TestEG 维护了一个中央数据库，记录所有任务的关键数据，如题目 ID、生成时间、参数设置、结果路径等信息。这种设计使得系统可以快速定位和检索特定条件下生成的测试数据，极大地提高了数据管理效率。

5.4 系统实现

TestEG 基于 BuctOJ 系统开发，前端使用 Vue 框架搭建，后端基于 Django 进行开发，数据库使用了 MySQL。系统开发的软件环境如表 5-1 所示。

表 5-1 系统软件环境
Table 5-1 System software environment

软件环境	版本
Vue	3.0
Node.js	14.17.5
Nginx	1.18.0
Django	3.2.13
Python	3.8.20
Redis	7.2.4
MySql	8.0.42

5.4.1 多维评估界面

多维评估界面主要包含代码覆盖率、变异得分、测试规模、假阳性程序和评估历史五大功能页面。前四个页面支持分别从不同维度对测试数据进行评估，评估历史页面则汇总了全部评估结果，方便用户进行管理操作。

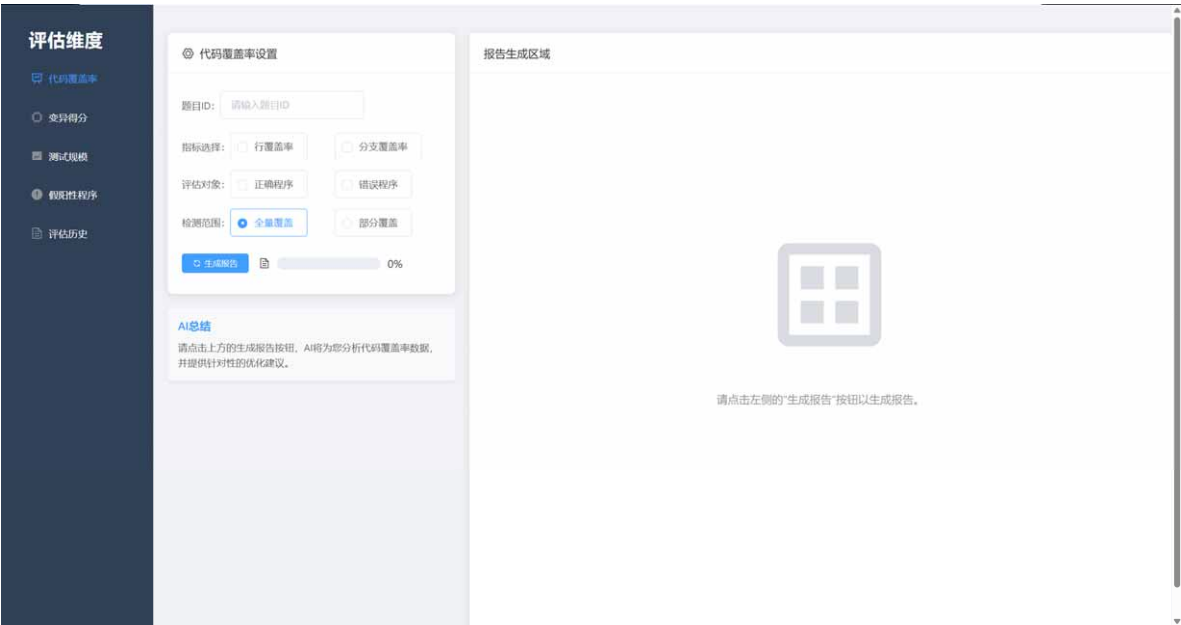


图 5-3 代码覆盖率评估页面
Fig.5-3 Code coverage evaluation page

图 5-3 展示了代码覆盖率页面，该页面主要由配置卡片、AI 总结和结果卡片三部分组成。在配置卡片中，用户可以选择具体的评价指标，如行覆盖率与分支覆盖率。由于 OJ 系统在长期的运行中积累了大量的学生程序，因此 TestEG 还提供了全量测试和部分测试的选择。用户可以选择测试全部的学生程序，或从中随机抽样 10%、20%、50%进行测试。此外，用户还可以选择仅评估正确程序集或错误程序集。

用户完成参数配置后,可以点击配置卡片下方的“生成报告”按钮进行评估。评估过程中,按钮上的文字会变成“评估中...”。此时前端系统携带参数配置发送 http 请求,后端系统接收请求后解析参数并向数据库中插入一条评估任务,再由任务处理线程拉取任务,调用相应的工具执行任务。任务执行完成后,处理线程会将评估结果存储在数据库中,并基于结果信息生成环状图存储在服务器指定路径中。在此过程中,前端系统通过轮询不断查询任务进展,并通过进度条更新反馈给用户。当获取到执行结果时,前端系统会在报告生成区域展示环状图信息,用户可以直观的看出该题目的测试数据在学生程序上的覆盖情况。此外,后端系统还会将结果信息与预设的提示词拼接输入给 LLM, LLM 在分析覆盖信息后给出问题分析和改进意见,并随环状图一起返回给前端,展示在 AI 总结卡片中。结果如图 5-4 所示。



图 5-4 评估结果页面

Fig.5-4 Evaluation result page



图 5-5 其余模块的参数设置页面

Fig.5-5 Parameter settings page for other modules

如图 5-5 所示,在变异得分评估页面,用户可以选择其检测范围,包括全量覆盖和部分覆盖。在测试规模评估页面,用可以选择评估指标,包括测试数据集、测试数

据行数和测试元素数。在假阳性程序评估页面，用户可以选择是否聚类以及差分测试所用算法，包括 SFDT 和 DT。

图 5-6 为历史评估记录页面，TestEG 使用表格展示了用户发起的所有评估任务。表格从左至右分别是序号、题目 ID、评估指标、结果路径、评估进度、开始时间、完成时间和相关操作。其中，结果路径存储了 TestEG 评估完成后生成的结果文件路径，评估进度包括“等待拉取”、“评估中”、“评估完成”和“取消评估”四种状态。当用户发起评估请求时，该评估任务会被插入数据库中，而如果此时任务处理线程均处于繁忙状态，那么任务便处于“拉取状态”。任务处理线程空闲时便会从数据库中拉取待处理的评估任务，此时任务状态修改为“评估中”，并在评估完成后将状态修改为“评估完成”。若用户误操作或需求变更后取消某次评估任务，则其状态转换为“取消评估”。

序号	题目ID	评估指标	结果路径	评估进度	开始时间	完成时间	操作
1	4254	代码覆盖率	/home/judge/src/evaluateResult/4254/cov.txt	100%	2025-02-...	2025-02-...	查看 下载 重新执行 删除
2	1070	代码覆盖率	/home/judge/src/evaluateResult/1070/cov.txt	100%	2025-02-...	2025-02-...	查看 下载 重新执行 删除
3	4254	测试用例	/home/judge/src/evaluateResult/4254/scale.txt	100%	2025-02-...	2025-02-...	查看 下载 重新执行 删除
4	4254	变异得分	/home/judge/src/evaluateResult/4254/mut.txt	100%	2025-02-...	2025-04-...	查看 下载 重新执行 删除
5	1069	代码覆盖率	/home/judge/src/evaluateResult/1069/cov.txt	100%	2025-02-...	2025-02-...	查看 下载 重新执行 删除
6	1069	测试用例	/home/judge/src/evaluateResult/1069/scale.txt	100%	2025-02-...	2025-02-...	查看 下载 重新执行 删除
7	1119	变异得分	/home/judge/src/evaluateResult/1119/mut.txt	100%	2025-02-...	2025-04-...	查看 下载 重新执行 删除
8	1070	变异得分	/home/judge/src/evaluateResult/1070/mut.txt	100%	2025-02-...	2025-04-...	查看 下载 重新执行 删除
9	1070	测试用例	/home/judge/src/evaluateResult/1070/scale.txt	100%	2025-03-...	2025-03-...	查看 下载 重新执行 删除
10	1119	测试用例	/home/judge/src/evaluateResult/1119/scale.txt	100%	2025-03-...	2025-03-...	查看 下载 重新执行 删除

图 5-6 评估历史记录页面
Fig.5-6 Evaluation history page

用户可以在该页面进行查看、下载、重新执行、取消和删除五种操作。查看功能允许用户跳转至该任务评估指标对应的页面中，查看评估结果和 AI 总结，同时 TestEG 会加载任务建立时用户选择的配置信息，还原历史评估场景。下载按钮允许用户将评估结果下载至本地查看、使用。重新执行使得用户可以在数据集或测试数据集更新后，重新以原先的配置进行评估任务。当评估任务未完成时，第四个按钮显示为取消按钮，用户可以终止该任务。当评估任务完成后，该按钮显示为删除按钮，供用户删除废弃的评估记录。

5.4.2 测试生成界面

测试生成界面主要包含三大功能页面：LLM 预生成、多目标优化和历史生成记录。其中，LLM 预生成和多目标优化协同工作，共同生成高质量测试数据集。历史生成记录模块则提供了高效的结果展示界面，方便用户进行管理操作。

图 5-7 展示了 LLM 预生成页面，该页面负责初始测试数据的生成。该页面主要由参数设置区、生成结果卡片和测试数据展示区域三部分组成。为了满足不同场景的生成需求，TestEG 集成了多种先进的 LLM，用户可以选择 DeepSeek R1、GPT-4o 和 Grok-3 等多种主流 LLM API。针对 LLM 的生成过程，用户可以对温度系数（temperature）、最大 token 数（Max Tokens）等关键参数进行调整，以控制 LLM 的创造性。参数配置下方为提示词编写框，TestEG 为用户内置了一套测试数据生成的提示词模板。用户可以在此基础上根据自身需求进行个性化修改。

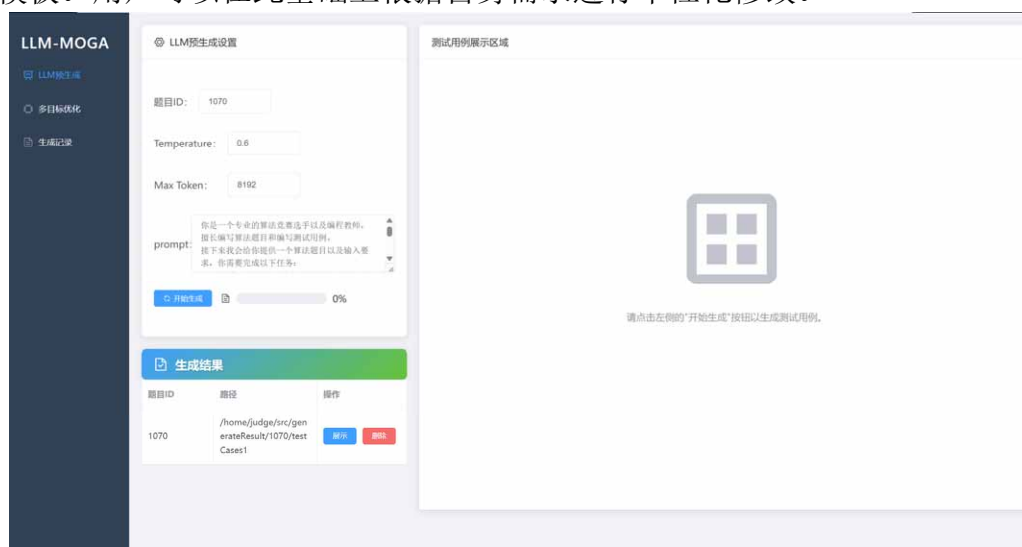


图 5-7 LLM 预生成界面

Fig.5-7 LLM Pre-Generation page

用户完成参数配置后可以点击“开始生成”按钮，按钮上的文字变成“生成中...”，右侧进度条会展示生成进度。此时前端系统会向后端任务发起任务请求，后端系统解析参数设置，并基于题目 ID 从数据库中拉取题目信息，与用户提交的提示词拼接形成结构化的完整提示，并调用所选的 LLM API 进行生成。此时，前端系统会轮询查询生成进度，并展示在进度条中。

此外，该生成记录会以表格的形式展示在结果卡片区域，包含题目 ID、结果存储路径、生成状态以及相关操作。生成状态包含“生成中”、“已完成”和“生成失败”三种状态，相关操作包含展示、重试、删除/取消三个按钮。如果任务执行失败，状态按钮展示“生成失败”，用户可以点击重试按钮让 TestEG 重新执行该生成任务，或点击取消按钮放弃此次生成任务。任务执行成功之后，用户可以点击展示按钮，此时测试数据展示区域会按文件结构有序呈现生成的测试数据集。由于部分测试数据的测试元素较多，TestEG 会将此类测试数据截取后展示，截取部分以省略号展示。生成结果

如图 5-8 所示。

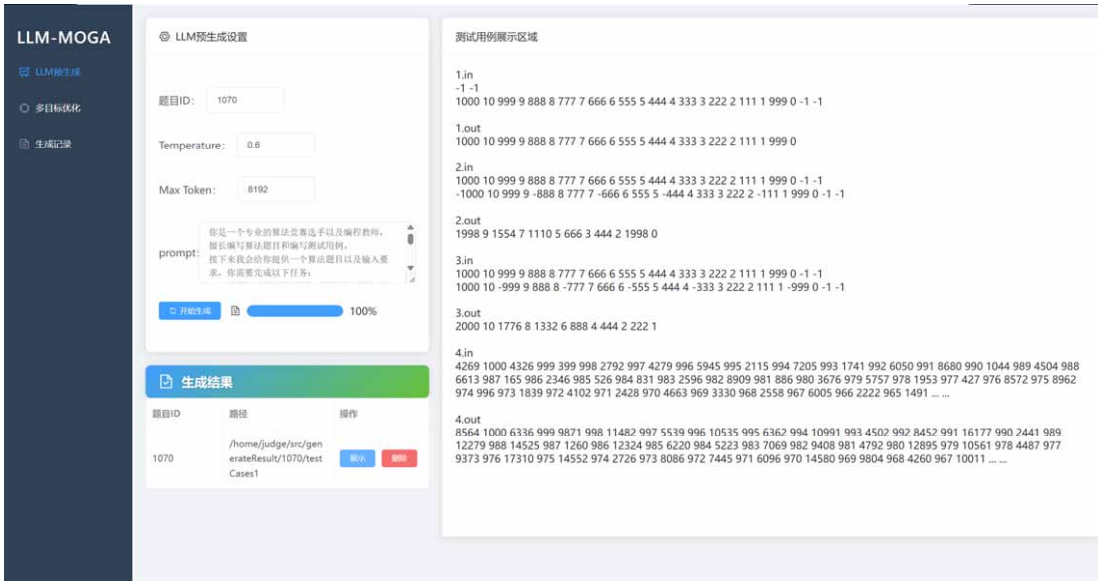


图 5-8 生成结果展示
Fig.5-8 Generation result page



图 5-9 多目标优化算法配置
Fig.5-9 Multi-Objective Optimization Algorithm Configuration

在多目标优化页面，如图 5-9 所示，TestEG 提供了丰富的算法参数配置接口，支持用户根据特定测试需求定制优化策略。用户可以设置交叉率与变异率来控制算法的收敛速度和搜索能力。TestEG 还支持多种适应度函数，包括代码覆盖率、错误检测率和测试规模，用户可以在每个指标前填写其权重值来调整适应度计算公式，从而引导测试数据的进化方向。若填写为 0，则在适应度计算中忽略该指标。此外，用户还可以在算法选择中挑选多目标优化算法，包括 SPEA-II 和 NSGA-II。

图 5-10 为历史生成记录页面，该页面提供了测试数据生成的追踪和管理功能。该页面以表格的形式记录了所有的生成任务，用户可以通过右上方的搜索框基于题目 ID 或生成状态来进行针对性检索。表格包含序号、题目 ID、生成状态、结果路径、生成进度、开始时间、完成时间和相关操作。其中生成状态包括“LLM 预生成”和“多目

标优化生成”两种状态，相关操作有查看、下载、优化和删除四个按钮。用户可以点击查看按钮跳转至该记录的生成状态所对应的页面，此页面记录了用户在发起此次生成任务时的全部参数配置，还原生成场景。用户可以点击下载按钮将生成的测试文件下载至本地查看。当生成状态为“LLM 预生成”时，用户可以点击优化按钮跳转至多目标优化页面，基于该初始种群进行优化。

序号	题目ID	生成状态	结果路径	生成进度	开始时间	完成时间	操作
1	1076	多目标优化生成	/home/judge/src/generateResult/1076	100%	2025-02-...	2025-02-...	查看 下载 优化 删除
2	1078	多目标优化生成	/home/judge/src/generateResult/1078	100%	2025-02-...	2025-03-...	查看 下载 优化 删除
3	1072	LLM预生成	/home/judge/src/generateResult/1072	100%	2025-02-...	2025-02-...	查看 下载 优化 删除
4	1071	LLM预生成	/home/judge/src/generateResult/1071	100%	2025-02-...	2025-02-...	查看 下载 优化 删除
5	1070	LLM预生成	/home/judge/src/generateResult/1070	100%	2025-03-...	2025-03-...	查看 下载 优化 删除
6	1079	多目标优化生成	/home/judge/src/generateResult/1079	100%	2025-03-...	2025-03-...	查看 下载 优化 删除
7	1077	多目标优化生成	/home/judge/src/generateResult/1077	100%	2025-03-...	2025-03-...	查看 下载 优化 删除
8	1075	多目标优化生成	/home/judge/src/generateResult/1075	100%	2025-03-...	2025-03-...	查看 下载 优化 删除
9	1073	多目标优化生成	/home/judge/src/generateResult/1073	100%	2025-03-...	2025-03-...	查看 下载 优化 删除
10	1074	多目标优化生成	/home/judge/src/generateResult/1074	100%	2025-03-...	2025-03-...	查看 下载 优化 删除

图 5-10 历史生成记录页面

Fig.5-10 Generation history page

5.5 本章小结

本章主要详细介绍了面向在线评测系统的测试数据生成与评估工具 TestEG。首先，本章节分析了 TestEG 的系统流程，介绍了前端、后端及数据库各自的职能与作用。随后详细介绍了 TestEG 的核心模块，描述了其设计与实现。最后详细介绍了 TestEG 的系统实现，并展示了系统页面及使用方式。该工具旨在便捷的评估和生成测试数据，为编程教学提供方便、高效的辅助工具。

第六章 结论与展望

6.1 研究工作总结

本文首先对 OJ 系统的算法题目与测试数据进行了分析,指出算法题目类型繁多、输入规模庞大等特征,导致人工设计全面、有效的测试数据难度剧增。该问题引发了 OJ 系统中的漏报现象,即学生程序中存在错误,但由于现有测试数据的错误检查能力有限,程序被误判为正确程序。这类程序被称为假阳性程序,严重影响了评测结果的准确性与系统的公正性。因此,本文提出了一种多维测试数据评估框架,旨在全面评估 OJ 系统中现有测试数据的质量。该框架使用了现有研究中最流行的代码覆盖率和变异得分两大指标,并结合 OJ 场景下测试数据辅助学生复现错误、修改 bug 的特征,提出了测试规模指标。本文从 BuctOJ 和 AtCoder 两大流行的编程网站上收集了 12 道算法题目,共计 33205 份学生程序进行了实验。实验结果表明,现有测试数据的平均行覆盖率仅为 81.71%,分支覆盖率仅为 79.35%,变异得分为 0.81。此外,大多数题目的平均测试元素数都超过了 1000。这些数据表明现有测试数据仍然存在较多缺陷。

为了检测 OJ 系统中的漏报情况,本文提出了一种基于聚类 and 差分测试的假阳性程序检测方法。该方法首先解析学生程序,提取所有的表达式语句。随后在表达式语句后插装输出语句,并执行测试数据来获取变量的值序列。基于此动态特征信息,本方法将海量的学生程序聚类至少量的族群中。接下来,本方法对题目信息和学生程序的静态特征进行分析,生成更易触发边界、分支、数据上限等漏洞的测试数据集。从族群中随机挑选代表程序在此测试数据集上进行差分测试,通过多数投票机制判定假阳性程序。该方法避免了因重复测试和测试冗余造成的资源浪费。实验结果表明,本方法有效约简了 54.57% 的学生程序,并从 14712 份正确程序中检测出了 2350 份假阳性程序。与 DT 方法相比,本方法减少了 50.39% 的时间开销,并在 Recall 和 F-measure 上取得了更好的效果。

针对现有测试数据存在的质量问题,本文提出了一种高质量的测试数据生成方法。本文首先分析了 LLM 和遗传算法在测试数据生成中的优势与劣势: LLM 具备卓越的语义理解和代码生成能力,但受到 token 和上下文的限制,难以直接生成高质量测试数据;遗传算法擅长迭代优化,搜索过程和结果会受到初始种群的影响。基于两者的优势,本文提出了一种基于大语言模型和多目标优化算法的测试数据生成方法。该方法通过提示词工程,利用 LLM 生成多样化的测试数据集。随后,以此测试数据集为初始种群,采用多目标优化算法进行迭代优化。在迭代过程中,本方法以错误检测率、代码覆盖路径和测试规模的加权和为适应度函数,旨在生成兼顾错误检测能力和测试

规模的高质量测试数据。为克服传统代码覆盖率忽视稀有路径的局限性，本文提出了一种基于覆盖频次的动态权重调整策略，通过给增加稀有路径的权重，引导算法覆盖更多的代码路径。实验结果表明，本方法生成测试数据的平均错误检测率达到了 97.62%，相比现有测试数据和 GA 方法生成的测试数据的错误检测率分别提升了 6.07% 和 6.77%，测试规模分别缩小了 6.64 倍和 5.05 倍。同时，在假阳性程序检测方面，相比于 AID 方法提升了 19.44% 的检测率。

最后，本文设计并实现了基于 B/S 架构的面向 OJ 系统的测试数据评估和生成工具 TestEG，该工具实现了本文提出的多维评估方法和测试数据自动生成技术，为编程教学提供了辅助功能。

6.2 展望

本文提出的假阳性程序检测方法和测试数据自动生成方法目前主要针对 C++ 程序设计，未来可考虑将其扩展至更多编程语言，如 Python 和 Java，以提升方法的通用性和适用范围。此外，本文表明，LLM 与多目标优化算法在测试数据生成中展现出较强的互补性。然而，目前本文仅利用 LLM 生成初始种群，未来可进一步探索将其融入多目标优化算法的搜索过程，例如通过提供更高质量的变异和交叉算子，增强搜索能力，从而在更复杂的算法题目中实现更丰富的路径覆盖和更高的生成效率。

参 考 文 献

- [1] Wasik S, Antczak M, Badura J, et al. A survey on online judge systems and their applications[J]. ACM Computing Surveys (CSUR), 2018, 51(1): 1-34.
- [2] Gulwani S, Radiček I, Zuleger F. Automated clustering and program repair for introductory programming assignments[J]. ACM SIGPLAN Notices, 2018, 53(4): 465-480.
- [3] Yi J, Ahmed U Z, Karkare A, et al. A feasibility study of using automated program repair for introductory programming assignments[C]//Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering. 2017: 740-751.
- [4] Francisco R E, Ambrosio A P. Mining an Online Judge System to Support Introductory Computer Programming Teaching[C]//EDM (Workshops). 2015.
- [5] Wu H, Liu Y, Qiu L, et al. Online judge system and its applications in c language teaching[C]//2016 International Symposium on Educational Technology (ISET). IEEE, 2016: 57-60.
- [6] Revilla M A, Manzoor S, Liu R. Competitive learning in informatics: The UVa online judge experience[J]. Olympiads in informatics, 2008, 2(10): 131-148.
- [7] Zinovieva I S, Artemchuk V O, Iatsyshyn A V, et al. The use of online coding platforms as additional distance tools in programming education[C]//Journal of physics: Conference series. IOP Publishing, 2021, 1840(1): 012029.
- [8] Zhu H, Hall P A V, May J H R. Software unit test coverage and adequacy[J]. Acm computing surveys (csur), 1997, 29(4): 366-427.
- [9] Weyuker E J. The evaluation of program-based software test data adequacy criteria[J]. Communications of the ACM, 1988, 31(6): 668-675.
- [10] Gopinath R, Jensen C, Groce A. Code coverage for suite evaluation by developers[C]//Proceedings of the 36th international conference on software engineering. 2014: 72-82.
- [11] Mutation testing for the new century[M]. Springer Science & Business Media, 2001.
- [12] Hemmati H. How effective are code coverage criteria?[C]//2015 IEEE International Conference on Software Quality, Reliability and Security. IEEE, 2015: 151-156.
- [13] Someoliayi K E, Jalali S, Mahdieh M, et al. Program state coverage: A test coverage metric based on executed program states[C]//2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER). IEEE, 2019: 584-588.
- [14] DeMillo R A, Lipton R J, Sayward F G. Hints on test data selection: Help for the practicing programmer[J]. Computer, 1978, 11(4): 34-41.
- [15] Papadakis M, Kintis M, Zhang J, et al. Mutation testing advances: an analysis and survey[M]//Advances in computers. Elsevier, 2019, 112: 275-378.

- [16] Papadakis M, Le Traon Y. Effective fault localization via mutation analysis: A selective mutation approach[C]//Proceedings of the 29th annual ACM symposium on applied computing. 2014: 1293-1300.
- [17] Papadakis M, Le Traon Y. Metallaxis - FL: mutation - based fault localization[J]. Software Testing, Verification and Reliability, 2015, 25(5-7): 605-628.
- [18] Liu Y, Li Z, Wang L, et al. Statement-oriented mutant reduction strategy for mutation based fault localization[C]//2017 IEEE International Conference on Software Quality, Reliability and Security (QRS). IEEE, 2017: 126-137.
- [19] de Oliveira A A L, Camilo-Junior C G, de Andrade Freitas E N, et al. Ftmes: A failed-test-oriented mutant execution strategy for mutation-based fault localization[C]//2018 IEEE 29th International Symposium on Software Reliability Engineering (ISSRE). IEEE, 2018: 155-165.
- [20] Wang H, Du B, He J, et al. Ietcr: An information entropy based test case reduction strategy for mutation-based fault localization[J]. IEEE Access, 2020, 8: 124297-124310.
- [21] Liu Y, Li Z, Zhao R, et al. An optimal mutation execution strategy for cost reduction of mutation-based fault localization[J]. Information Sciences, 2018, 422: 572-596.
- [22] Anand S, Burke E K, Chen T Y, et al. An orchestrated survey of methodologies for automated software test case generation[J]. Journal of systems and software, 2013, 86(8): 1978-2001.
- [23] Aniche M, Treude C, Zaidman A. How developers engineer test cases: An observational study[J]. IEEE Transactions on Software Engineering, 2021, 48(12): 4925-4946.
- [24] Bertolino A. Software testing research: Achievements, challenges, dreams[C]//Future of Software Engineering (FOSE'07). IEEE, 2007: 85-103.
- [25] King J C. A new approach to program testing[J]. ACM Sigplan Notices, 1975, 10(6): 228-233.
- [26] Avgerinos T, Rebert A, Cha S K, et al. Enhancing symbolic execution with veritesting[C]//Proceedings of the 36th International Conference on Software Engineering. 2014: 1083-1094.
- [27] Cadar C, Dunbar D, Engler D R. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs[C]//OSDI. 2008, 8: 209-224.
- [28] Cadar C, Godefroid P, Khurshid S, et al. Symbolic execution for software testing in practice: preliminary assessment[C]//Proceedings of the 33rd International Conference on Software Engineering. 2011: 1066-1071.
- [29] Khurshid S, Păsăreanu C S, Visser W. Generalized symbolic execution for model checking and testing[C]//International Conference on Tools and Algorithms for the Construction and Analysis of Systems. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003: 553-568.

-
- [30] Krishnamoorthy S, Hsiao M S, Lingappan L. Tackling the path explosion problem in symbolic execution-driven test generation for programs[C]//2010 19th IEEE Asian Test Symposium. IEEE, 2010: 59-64.
- [31] Ma K K, Yit Phang K, Foster J S, et al. Directed symbolic execution[C]//Static Analysis: 18th International Symposium, SAS 2011, Venice, Italy, September 14-16, 2011. Proceedings 18. Springer Berlin Heidelberg, 2011: 95-111.
- [32] Godefroid P, Taly A. Automated synthesis of symbolic instruction encodings from I/O samples[J]. ACM SIGPLAN Notices, 2012, 47(6): 441-452.
- [33] Anand S, Harrold M J. Heap cloning: Enabling dynamic symbolic execution of java programs[C]//2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011). IEEE, 2011: 33-42.
- [34] Borges M, d'Amorim M, Anand S, et al. Symbolic execution with interval solving and meta-heuristic search[C]//2012 IEEE Fifth International Conference on Software Testing, Verification and Validation. IEEE, 2012: 111-120.
- [35] Souza M, Borges M, d'Amorim M, et al. Coral: Solving complex constraints for symbolic pathfinder[C]//NASA Formal Methods: Third International Symposium, NFM 2011, Pasadena, CA, USA, April 18-20, 2011. Proceedings 3. Springer Berlin Heidelberg, 2011: 359-374.
- [36] Fraser G, Arcuri A. Evolutionary generation of whole test suites[C]//2011 11th International Conference on Quality Software. IEEE, 2011: 31-40.
- [37] Fraser G, Arcuri A. Evosuite: automatic test suite generation for object-oriented software[C]//Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering. 2011: 416-419.
- [38] Avila-George H, Torres-Jimenez J, Gonzalez-Hernandez L, et al. Metaheuristic approach for constructing functional test - suites[J]. IET software, 2013, 7(2): 104-117.
- [39] Sahin O, Akay B. Comparisons of metaheuristic algorithms and fitness functions on software test data generation[J]. Applied Soft Computing, 2016, 49: 1202-1214.
- [40] Dakhel A M, Nikanjam A, Majdinasab V, et al. Effective test generation using pre-trained large language models and mutation testing[J]. Information and Software Technology, 2024, 171: 107468.
- [41] Yuan Z, Lou Y, Liu M, et al. No more manual tests? evaluating and improving chatgpt for unit test generation[J]. arXiv preprint arXiv:2305.04207, 2023.
- [42] Schäfer M, Nadi S, Eghbali A, et al. Adaptive test generation using a large language model[J]. arXiv preprint arXiv:2302.06527, 2023: 1-21.

- [43] Jorgensen S, Nadizar G, Pietropolli G, et al. Large Language Model-based Test Case Generation for GP Agents[C]//Proceedings of the Genetic and Evolutionary Computation Conference. 2024: 914-923.
- [44] Deb K. Multi-objective optimisation using evolutionary algorithms: an introduction[M]//Multi-objective evolutionary optimisation for product design and manufacturing. London: Springer London, 2011: 3-34.
- [45] Zhou A, Qu B Y, Li H, et al. Multiobjective evolutionary algorithms: A survey of the state of the art[J]. Swarm and evolutionary computation, 2011, 1(1): 32-49.
- [46] Wagner M, Friedrich T. Efficient parent selection for approximation-guided evolutionary multi-objective optimization[C]//2013 IEEE Congress on Evolutionary Computation. IEEE, 2013: 1846-1853.
- [47] Matnei Filho R A, Vergilio S R. A mutation and multi-objective test data generation approach for feature testing of software product lines[C]//2015 29th Brazilian Symposium on Software Engineering. IEEE, 2015: 21-30.
- [48] Pradhan D, Wang S, Ali S, et al. Employing rule mining and multi-objective search for dynamic test case prioritization[J]. Journal of Systems and Software, 2019, 153: 86-104.
- [49] Deb K, Pratap A, Agarwal S, et al. A fast and elitist multiobjective genetic algorithm: NSGA-II[J]. IEEE transactions on evolutionary computation, 2002, 6(2): 182-197.
- [50] Zitzler E, Laumanns M, Thiele L. SPEA2: Improving the strength Pareto evolutionary algorithm[J]. TIK report, 2001, 103.
- [51] Malaiya Y K, Li M N, Bieman J M, et al. Software reliability growth with test coverage[J]. IEEE Transactions on Reliability, 2002, 51(4): 420-426.
- [52] Aghamohammadi A, Mirian-Hosseiniabadi S H, Jalali S. Statement frequency coverage: a code coverage criterion for assessing test suite effectiveness[J]. Information and Software Technology, 2021, 129: 106426.
- [53] <http://www.agitar.com/>
- [54] <http://dynamic-memory.com/>
- [55] Delgado-Pérez P, Medina-Bulo I, Palomo-Lozano F, et al. Assessment of class mutation operators for C++ with the MuCPP mutation system[J]. Information and Software Technology, 2017, 81: 169-184.
- [56] Gopinath R, Alipour A, Ahmed I, et al. How hard does mutation analysis have to be, anyway?[C]//2015 IEEE 26th International Symposium on Software Reliability Engineering (ISSRE). IEEE, 2015: 216-227.
- [57] Jia Y, Harman M. Higher order mutation testing[J]. Information and Software Technology, 2009, 51(10): 1379-1393.

-
- [58] Papadakis M, Malevris N. An empirical evaluation of the first and second order mutation testing strategies[C]//2010 Third International Conference on Software Testing, Verification, and Validation Workshops. IEEE, 2010: 90-99.
- [59] Kintis M, Papadakis M, Malevris N. Evaluating mutation testing alternatives: A collateral experiment[C]//2010 Asia Pacific Software Engineering Conference. IEEE, 2010: 300-309.
- [60] Tufano M, Kimko J, Wang S, et al. Deepmutation: A neural mutation tool[C]//Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings. 2020: 29-32.
- [61] Liu K, Han Y, Zhang J M, et al. Who judges the judge: An empirical study on online judge tests[C]//Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. 2023: 334-346.
- [62] Castro M, Costa M, Martin J P. Better bug reporting with better privacy[J]. ACM SIGOPS Operating Systems Review, 2008, 42(2): 319-328.
- [63] Santelices R, Chittimalli P K, Apiwattanapong T, et al. Test-suite augmentation for evolving software[C]//2008 23rd IEEE/ACM International Conference on Automated Software Engineering. IEEE, 2008: 218-227.
- [64] Majumdar R, Saha I. Symbolic robustness analysis[C]//2009 30th IEEE Real-Time Systems Symposium. IEEE, 2009: 355-363.
- [65] Srivastava P R, Ramachandran V, Kumar M, et al. Generation of test data using meta heuristic approach[C]//TENCON 2008-2008 IEEE Region 10 Conference. IEEE, 2008: 1-6.
- [66] Hashim F A, Houssein E H, Hussain K, et al. Honey Badger Algorithm: New metaheuristic algorithm for solving optimization problems[J]. Mathematics and Computers in Simulation, 2022, 192: 84-110.
- [67] Lemieux C, Inala J P, Lahiri S K, et al. Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models[C]//2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, 2023: 919-931.
- [68] Deng Y, Xia C S, Peng H, et al. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models[C]//Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis. 2023: 423-435.
- [69] Li T O, Zong W, Wang Y, et al. Nuances are the key: Unlocking chatgpt to find failure-inducing tests with differential prompting[C]//2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, 2023: 14-26.
- [70] Jones J A, Harrold M J, Stasko J. Visualization of test information to assist fault localization[C]//Proceedings of the 24th international conference on Software engineering. 2002: 467-477.

- [71] 余德利. 面向错误定位的程序在线评测系统测试数据优化方法研究[D]. 北京化工大学, 2021.
- [72] Maaranen H, Miettinen K, Penttinen A. On initial populations of a genetic algorithm for continuous optimization problems[J]. *Journal of Global Optimization*, 2007, 37: 405-436.
- [73] Liu Y, Liu H, Yang Z, et al. Empirical Evaluation of Large Language Models for Novice Program Fault Localization[C]//2024 IEEE 24th International Conference on Software Quality, Reliability and Security (QRS). IEEE, 2024: 180-191.
- [74] Li Z, Zhou X, Yu D, et al. Fault localization-guided test data generation approach for novice programs[C]//2021 8th International Conference on Dependable Systems and Their Applications (DSA). IEEE, 2021: 54-65.
- [75] Liu K, Liu Y, Chen Z, et al. Llm-powered test case generation for detecting tricky bugs[J]. *arXiv preprint arXiv:2404.10304*, 2024.